
Final Year Project

HEPA-M: Hierarchical Energy- and Path-Aware Microservice Migration Framework

Student James Finn

A thesis submitted in part fulfilment of the degree of

BSc. (Hons.) in Computer Science

Supervisor: Professor Liam Murphy



UCD School of Computer Science
University College Dublin

April 28, 2026

Table of Contents

1	Introduction	4
1.1	Project Objectives	5
2	Related Work and Ideas	6
2.1	Cloud Computing	6
2.2	Internet of Things (IoT)	6
2.3	Multi-Access Edge Computing (MEC)	7
2.4	Quality of Experience (QoE)	8
2.5	Edge Service Migration	8
2.6	Server Utilisation and Performance	9
2.7	Energy Consumption in Edge Computing	9
2.8	iFogSim2 Simulator	10
2.9	Case Study Application	10
3	Design Considerations and Implementation	12
3.1	Overview of HEPA-M	12
3.2	System Architecture	13
3.3	Path-Based Infrastructure Model	13
3.4	Limitations of Default iFogSim2 Migration Behaviour	14
3.5	Path-Aware Module Placement	15
3.6	Resource-Aware Gateway Selection	17
3.7	Dynamic Microservice Migration within the Infrastructure Hierarchy	19
3.8	Evaluation Metrics	19
4	Simulation Results and Analysis	22
4.1	Behaviour Simulations	23
4.2	Performance Simulations	35
5	Conclusion and Future Work	42
5.1	Conclusion	42

5.2	Future Work	42
6	Appendix	45
6.1	Revised Default Mobility Controller Code	45
6.2	Dynamic Mobility Controller Code	52
6.3	Path-Aware Module Placement	59
6.4	Resource-Aware Gateway Selection	60

Abstract

Multi-Access Edge Computing (MEC) is an architecture that enables low latency processing for Internet of Things (IoT) applications, through the deployment of computing at the "edge" near the user. However, achieving this low latency objective is challenging within a mobile environment due to the fact that IoT application microservices will be migrated between edge nodes as the user moves.

Current migration strategies, including those applied in iFogSim2, typically use proximity as the primary criteria for determining a target node, and limit microservice placement to a specific path. These limitations can result in non-uniform resource allocation, over-migration of microservices to higher network layers, and unpredictable increases in latency.

This project adds a resource-aware migration framework to iFogSim2, designed to enhance both the latency stability and energy efficiency of MEC environments. A threshold-based gateway selection strategy has been developed to consider CPU utilisation when selecting a new parent node, which allows active gateways to be given priority over passive ones and minimises the activation of additional infrastructure. Furthermore, microservice placement restrictions have been removed to allow microservices to be placed closer to users when resources allow it. Additionally, microservices can now dynamically migrate up and down the network hierarchy based on CPU load as a result of user mobility events.

The simulation results show that these enhancements have improved the distribution of workload among the edge nodes, prevented large latency spikes during workload changes, resulted in significantly reduced average end-to-end latency, while also providing configurable trade-off capabilities to control energy consumption versus performance. HEPA-M provides a more adaptive and resource-aware approach to migrating services for MEC systems that support latency-sensitive IoT applications.

Chapter 1: Introduction

The adoption of cloud computing has seen a major increase recently. It provides users access to a wide variety of services and applications that can be accessed from centralised data centres. A significant issue regarding the usage of cloud computing is the physical distance between the cloud data centre and the consumer, resulting in network latency. Network latency is becoming an ever more prominent issue as the number of devices on the Internet of Things (IoT) grows exponentially. IoT systems produce large amounts of data and compete for limited bandwidth in networks, making it very difficult for centralised cloud architectures to deliver consistently low response times. Multi-Access Edge Computing (MEC) has emerged as a paradigm which deploys computational resources closer to the user by placing them at the network edge. The reduction in distance results in lower latency when transmitting data and an improved responsive experience for the user.

Although MEC provides many benefits, there are still issues with managing distributed infrastructures. Edge deployments involve multiple nodes in the same geographical area, each capable of supporting multiple types of applications and workloads. Additionally, many IoT systems have real-time or near-real-time requirements, requiring low latency in their operation. Systems such as cardiovascular health monitoring systems, which my simulation mimics require rapid processing of sensor data to function properly. Thus, maintaining low and stable latency for applications of this nature is critical.

Another concern in relation to large-scale edge deployments is the large quantity of energy being consumed. Low-latency applications require multiple edge nodes to remain active to ensure the service remains close to the user. As the number of deployed devices grows, so does the total energy footprint of the entire system. Therefore, balancing low latency with the efficient utilisation of computing resources presents a significant challenge for researchers and industry professionals alike.

User mobility adds another layer of complexity to the challenges associated with MEC. As users move throughout the network, the microservices supporting them will need to move between edge nodes to ensure acceptable latency levels. This creates two primary questions for mobility management systems: 1) when should the microservices migrate; and 2) where would be the best place to migrate to? Most current approaches rely on simple heuristics to determine the best destination. For instance, the iFogSim2 simulator implements a default migration mechanism that chooses the user's nearest available gateway based solely on its proximity to the user and not on the current load of the gateway. Although this method is easy to implement, it can result in unevenly distributed loads and thus reduced performance when nearby gateways are highly utilised.

Previous work explored resource-aware migration techniques to address some of the shortcomings of previous approaches. Specifically, the DRAMA algorithm developed by Noemi introduced a migration approach that considered available resources when selecting the next parent node during a user mobility event. Although this represents a significant improvement over proximity-based migration approaches, there are still limitations when using current simulation models to replicate realistic behaviour in MEC environments.

This project builds upon the challenges identified above and seeks to create an extended version of the iFogSim2 simulator that includes a more adaptive migration and microservice management framework. HEPA-M includes a resource-aware gateway selection mechanism that considers the current CPU utilisation of candidate gateways when determining the destination gateway for migrating microservices during user mobility events. Unlike the proximity-based migration

mechanisms currently implemented in simulation frameworks, this mechanism gives preference to gateways that are operating below a configurable utilisation threshold, while allowing additional infrastructure to be activated if necessary. This enables the exploration of trade-offs between latency performance and energy consumption.

Additionally, the project includes improvements to the behaviour in which microservices are managed within the infrastructure hierarchy. HEPA-M allows multiple instances of the same microservice to exist along the same infrastructure path. This enables microservices to be placed closer to the gateways servicing users when sufficient resources are available. The placement of microservices closer to the gateways allows services to be relocated from higher-level nodes when possible, thereby improving overall latency. Additionally, microservices can migrate dynamically within the infrastructure hierarchy based on changes in computational load, allowing the system to adapt to varying workloads and user mobility.

Overall, HEPA-M seeks to improve the distribution of workload across edge nodes while keeping microservices as close to users as possible. The behaviour of the system will be evaluated through simulation experiments, analysing how the utilisation threshold affects latency and energy consumption within MEC environments.

The remainder of this thesis is organised as follows. First, background information about the MEC paradigm and the challenges of mobility and energy consumption will be provided. Then, the system design and the modifications made to the iFogSim2 simulator will be described. Following that, the results of simulation experiments will be analysed to assess the behaviour and performance of HEPA-M. Finally, this thesis will conclude with a discussion of the contributions of this work and directions for future research.

1.1 Project Objectives

1. To develop a thorough understanding of the iFogSim2 simulator and codebase.
2. To extend the capabilities of the iFogSim2 framework to enable the proposed changes:
 - (a) Implement a resource-aware gateway selection algorithm with configurable energy-performance trade-offs.
 - (b) Implement improved microservice placement, allowing each microservice to be placed closer to the edge and enabling dynamic migration behaviour.
3. To test the proposed migration technique using simulations and assess its ability to improve latency and stability, reduce energy usage, and improve overall system behaviour.

Chapter 2: Related Work and Ideas

2.1 Cloud Computing

Cloud computing has developed into the prevailing paradigm for providing services over the internet. The National Institute of Standards and Technology (NIST) defines cloud computing as "model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources ... that can be rapidly provisioned and released with minimal management effort or service provider interaction" [1]. Computing resources typically comprise of networks, servers, storage, applications, and services located in massive-scale data centres, and then accessed by users via the internet.

The key advantage of cloud computing lies in its elasticity, it provides scalable and flexible computing resources without forcing users to purchase and maintain their own infrastructure. Cloud service providers use virtualisation and resource pooling to provide computing resources based on demand. Therefore, organisations can scale their computing resources efficiently while maintaining lower operating costs [2]. Additionally, this model allows for reduced complexity related to the maintenance and deployment of infrastructure, as organisations can outsource the management of their IT infrastructures to cloud service providers.

Although cloud computing offers many benefits, it has limitations that exist, particularly when organisations need low latency and/or near real time responses from applications. The physical distance between the user and the cloud data centre causes network latency, which can negatively affect the performance of latency sensitive applications [3]. Furthermore, the continuous growth of the IoT will result in an increased volume of data being transferred between devices and cloud servers; therefore, the strain on network bandwidth required to support data transmission will increase, and the delay associated with transmitting data will grow [4]. Thus, the above issues limit the applicability of traditional central cloud architectures to applications such as real time health monitoring, autonomous systems, and other time critical applications.

As a result of the limitations of traditional cloud computing, alternative computing models have been developed to enable the processing of data closer to users. One of these models is known as Multi-Access Edge Computing (MEC). In this paradigm, computational resources are deployed close to the edge of the network to minimise latency and improve response times. While MEC builds upon the foundational concepts of cloud computing, it attempts to overcome some of its major limitations, particularly those affecting latency-sensitive environments [5].

2.2 Internet of Things (IoT)

The Internet of Things (IoT) describes a collection of physical devices that are capable of exchanging data with other devices via the internet. IoT includes devices such as actuators and sensors, which are capable of interacting with both the physical environment and digital devices. As a result, a variety of IoT-based technologies have been developed to enable a range of applications in fields such as healthcare, smart city management, automated manufacturing, and transportation

systems [6].

A major characteristic of IoT systems is their ability to generate enormous volumes of data from the many devices connected within the Internet of Things. Data collected by IoT-based devices typically needs to be transmitted to remote cloud infrastructure for processing and storage. However, transmitting data from large numbers of distributed devices to remote cloud data centres can introduce communication delays and increase network congestion [7].

Additionally, many IoT-based applications require response times close to real time. For instance, medical monitoring systems and autonomous vehicles require rapid data processing to ensure safety and reliability. This means that for these types of applications, the latency associated with centralising data processing at remote cloud computing facilities could negatively affect application performance and user experience [8]. As a result of the constraints described above, there has been growing interest in computing paradigms that bring processing closer to end users, such as Multi-Access Edge Computing (MEC).

2.3 Multi-Access Edge Computing (MEC)

The use of multi-access edge computing (MEC), a computing paradigm, provides a means of enhancing cloud capabilities by placing computing resources closer to consumers at the network edge. Thus, instead of having all of their cloud capabilities based at remote data centres, MEC provides a means to deploy server functionality into consumer-facing networks. For example, MEC may be deployed into access networks such as base stations, gateways, or edge nodes. The deployment of server functionality into consumer-facing networks reduces the physical distance between consumers and computing resources. Therefore, MEC has the potential to significantly reduce the latency associated with the communication between consumer devices and cloud-based applications. Additionally, MEC has the potential to increase the responsiveness of consumer devices to applications that rely on low-latency communications [5].

IoT systems are well suited for MEC environments because they consist of a large number of devices that continuously produce streams of data requiring rapid processing. When computation is performed at the edge of the network (i.e., closer to where data was produced), less data needs to be transmitted back to the cloud infrastructure than would occur if the data were transmitted directly from the device to a central cloud infrastructure. Reduced data transmission requirements decrease the likelihood of network congestion, thereby increasing the efficiency of the overall system [9]. These characteristics make MEC an attractive solution for a variety of applications including real-time healthcare monitoring, autonomous vehicles, augmented reality, and industrial automation.

However, there are additional challenges associated with MEC infrastructure regarding the distributed nature of its computing resources. Specifically, edge nodes will generally have fewer computing resources available to them than larger cloud data centres and therefore must share those resources among multiple users and services. Furthermore, because edge nodes are located at the network edge, users may move between locations, causing services to migrate between different edge nodes in order to remain within an acceptable latency range. Therefore, the development of efficient resource management strategies and service migration techniques is necessary to ensure both performance and quality of experience (QoE) for MEC-based applications [10].

2.4 Quality of Experience (QoE)

Quality of Experience (QoE) is defined as the overall level of satisfaction experienced by a user when interacting with an application or service. While Quality of Service (QoS) traditionally measures performance characteristics related to networks and systems, QoE focuses on the end-user's evaluation of the performance of a system [11]. As a result, there are numerous technical factors that influence QoE; these include latency, bandwidth availability, processor speed, and other system resources [12]. In many IoT/MEC-based applications, one of the most significant technical factors influencing QoE is latency. Since many IoT/MEC applications rely on real-time data exchanges (e.g., real-time health monitoring, online gaming, autonomous systems), consistently low latency is required for these applications to function effectively. As latency increases, users' perceptions of their experiences can decrease dramatically even if other system resources remain available [13].

System stability is another factor that affects QoE. In a distributed environment (e.g., MEC infrastructure), it is common to observe sudden increases in workloads that can cause unpredictable variations in both latency and system resource utilisation. Consequently, systems that provide consistent and predictable behaviour as workload increases are preferable, as they enable applications to continue functioning reliably during periods of varying demand. Therefore, ensuring that latency and resource utilisation increase in a controlled and predictable manner as workload increases is important for achieving high QoE in MEC environments [14].

2.5 Edge Service Migration

The term edge service migration is used to describe the process of moving an active edge service or application from one compute node to another within a distributed computing architecture. MEC systems often require that edge services migrate with users as they transition between different network locations. This is because acceptable latency can only be maintained if services continue to run in proximity to the end-user [15].

In many MEC architectures, edge services are provisioned on edge nodes that are distributed across the network. A common problem arises when users move to a location that is farther from the original edge node providing the service. As a result, latency between the user and the service may increase. To maintain adequate low-latency communication and quality of experience (QoE), the service may need to migrate to an edge node located closer to the user [16].

Service migration introduces several challenges. For example, migration decision making must consider factors such as network latency, computational resource availability, and the cost associated with migrating services between nodes. Additionally, once it has been determined when a service should be migrated, determining where the service should migrate to is the next step. If services are migrated too frequently, the overhead introduced by these migrations can negatively affect system performance. However, if services are migrated too infrequently or to a location that is not optimal, latency between the user and the service may increase, ultimately reducing QoE [17]. Therefore, developing effective migration strategies is essential for maintaining an optimised MEC environment.

2.6 Server Utilisation and Performance

Server utilisation is a key factor in deciding how well MEC environments will perform. The primary function of the edge node in a MEC environment is to process application tasks and manage the data being created by connected devices. As the amount of computation being performed on a given edge node increases, so does the use of the edge node's available resources, i.e., CPU and Memory. At high levels of resource utilisation, the time to process tasks can increase, causing increased computational latency [18].

High levels of resource utilisation have been proven to significantly reduce the performance of edge nodes. As system loads increase, tasks experience longer processing times because they are competing for limited resources. The impact of system resource limitations is generally more significant in distributed MEC environments, as compared to large cloud data centres, since edge nodes contain far fewer computational resources in comparison [19].

To ensure consistent performance from MEC systems, it is crucial to maintain balance in the way resources are used among edge nodes. An unbalanced workload distribution throughout the network could create situations where some edge nodes are fully loaded while other edge nodes are idle or lightly loaded. Unbalanced resource utilisation creates additional latency and negatively affects Quality of Experience (QoE) for end users. Therefore, methods to effectively distribute workloads and manage resources are needed to help ensure that MEC environments continue to operate efficiently [20].

2.7 Energy Consumption in Edge Computing

Energy use is another major concern in modern computing infrastructures as the number of cloud and edge servers being deployed continues to grow. Although each edge device usually consumes much less energy than a large cloud-based data centre, the large number of edge devices required to provide the backbone of a MEC network can result in very high overall energy consumption [21].

A server's energy consumption depends heavily on the utilisation of its hardware components, especially the central processing unit (CPU) [22]. Research has shown that even when a server is idle, it may still consume a large percentage of its peak energy usage, often exceeding 70% of its maximum power consumption [23]. As CPU utilisation increases, the energy consumption of the server also increases. Resource utilisation-based energy models of servers demonstrate a direct relationship between increasing computational load and increasing energy consumption [24].

In addition, some energy models use non-linear equations to represent how energy consumption increases at a faster rate as utilisation approaches higher levels. This type of model more accurately reflects the behaviour of modern processor technology, where increasing the workload on a processor can cause a larger-than-proportional increase in energy consumption due to factors such as voltage scaling and thermal effects [25]. Thus, managing resource utilisation across active nodes in a MEC network is important both for maintaining performance and for reducing unnecessary energy consumption.

Increasing resource utilisation across active nodes can help reduce the total number of servers required to support application workloads. A smaller number of active nodes can host a given workload and avoid activating additional infrastructure, thereby reducing overall energy consumption without negatively affecting performance. Therefore, developing energy-aware resource man-

agement techniques is becoming increasingly important for building scalable and sustainable MEC networks.

2.8 iFogSim2 Simulator

iFogSim2 is a simulation framework designed for modelling and evaluating resource management strategies in IoT, fog, and edge computing environments. It is built upon the CloudSim simulation toolkit, which was originally developed for modelling cloud computing infrastructures and resource provisioning policies [26]. The original iFogSim framework extended CloudSim by introducing support for IoT and fog computing architectures. iFogSim2 further enhances this framework by adding features such as microservice orchestration, clustering mechanisms, and mobility support within distributed edge environments [27].

Within the iFogSim2 simulation environment, several components are used to represent the behaviour of IoT and edge computing systems. Sensors are commonly used to represent devices that generate data from the physical environment, while actuators represent components that perform actions based on processed data. Data produced within the system is represented in the simulator as tuples, which are transmitted through the network for processing. These tuples are processed by application modules deployed on FogDevice nodes, which represent the computational resources available across the cloud, fog, and edge layers of the infrastructure. Although sensors and actuators are commonly used in many IoT simulations, their use depends on the specific application being modelled within the simulator.

Applications in iFogSim2 are typically modelled using a microservice-based architecture. In this approach, application functionality is divided into smaller processing modules that communicate through data flows represented by tuples. These modules can be deployed on different nodes across the network hierarchy, allowing researchers to evaluate how different service placement strategies affect system performance.

iFogSim2 also includes support for modelling user mobility within edge computing environments. As users move through the network, the simulator can trigger service migration events to maintain application responsiveness and Quality of Experience (QoE). This capability allows researchers to study the impact of different migration and resource management strategies in dynamic MEC environments. Due to its flexibility and support for modelling IoT systems, microservice architectures, and user mobility, iFogSim2 is widely used as a research platform for studying edge and fog computing systems [26].

2.9 Case Study Application

To assess how HEPA-M behaves, a health monitoring application using continuous ECG monitoring was selected for use in the simulated environment as the test application [27]. Applications utilising wearable health monitors to continuously collect physiological data produce an ongoing flow of data which requires nearly instantaneous processing to identify possible anomalies and provide the user with real-time feedback. Therefore, these types of applications are good candidates for testing within Mobile Edge Computing (MEC) environments due to the potential to deploy services at the edge, closer to the user, thereby reducing latency.

The application utilises a microservice-based architecture including a client module and three backend microservices. Each of the microservices has one primary function with respect to the data processing pipeline:

- **clientModule** – Located on the user’s device, the client module serves as the interface to the end-user, collects ECG data from the sensor, sends the collected data to mService1 for processing, and then returns the results to the end-user via the display actuator.
- **mService1** – Provides the first level of preprocessing to the ECG data collected by the sensor, including removing noise and preparing the signal for additional processing performed by mService2.
- **mService2** – Performs latency-critical monitoring to immediately recognise abnormal conditions and generate rapid responses or alerts for the user.
- **mService3** – Provides predictive analytics, detects trends over time, and performs more complex analysis. Due to its higher resource utilisation and lower sensitivity to latency compared to mService2, this service is deployed in the Cloud layer.

Due to the separation of latency-sensitive and high resource utilisation services, the application allows for the evaluation of both service placement and migration behaviour in MEC environments.

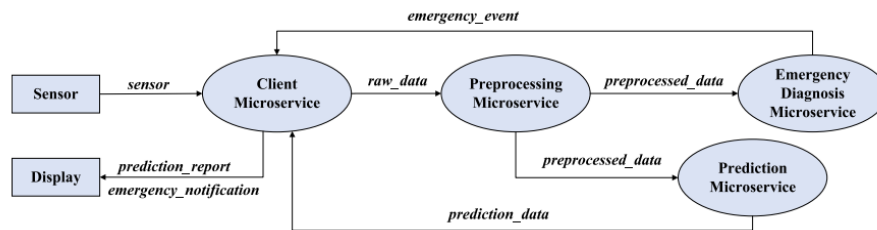


Figure 2.1: Microservice application model

Chapter 3: Design Considerations and Implementation

3.1 Overview of HEPA-M

This chapter will outline the design principles for the framework created to enhance service placement and migration behaviour in the context of MEC environments. To implement HEPA-M, the default iFogSim2 mobility and placement components were extended to support resource-aware gateway selection, path-aware module placement, and hierarchical microservice migration.

In the original iFogSim2 design:

- `MobilityController` manages the simulation by coordinating device interactions, application deployment, and event execution across the fog environment. `MobilityController` extends `Controller` to support mobility events. This class contained logical migration issues explained in Section 3.4.
- `LocationHandler` references `DataParser` which is responsible for user mobility locations and determines new parent gateway during migrations.
- `ModulePlacementMobileEdgewards` performs the initial microservice placement, placing services as close to the edge as feasible.

HEPA-M introduces the following class extensions:

- `MobilityControllerDynamic`: Extends the default mobility controller, in addition to handling user movement, it allows for hierarchical migration. When computational load increases at lower layers of the network, microservices may be migrated upward to higher-capacity nodes to prevent overload. Conversely, when resources become available closer to the edge, services may be migrated downward to remain closer to end users and maintain low latency ??.
- `LocationHandlerEDRAMA`: Extends the default location handler to support resource-aware gateway selection. Based on Noemi's DRAMA Algorithm, it considers the available resources of a device's associated path when selecting a destination gateway. Furthermore, a configurable threshold for the utilisation of gateways has been added to influence how gateways are selected and activated. Allowing us to balance energy consumption and performance as discussed in Section 3.6.
- `ModulePlacementDynamicMobileEdgewards`: Extends the original placement strategy to support path-aware module placement throughout the simulation. Unlike the original placement strategy which is only used during initial placement, the extended version is used throughout the entire simulation 3.5.

Together, these extensions integrate dynamic mobility handling, resource-aware gateway selection, and path-aware microservice placement into the iFogSim2 environment.

3.2 System Architecture

In order to reflect an MEC environment, the system architecture follows the hierarchical structure defined by the iFogSim2 simulator, which is typical of Multi-Access Edge Computing (MEC). The architecture has four layers: the cloud layer is positioned furthest away from the users and represents the highest level of computing power; the proxy layer represents the intermediate coordination point for a group of gateways located in a specific geographic area; the gateway layer represents the edge devices that can host microservices and communicate directly with mobile users; and the final layer consists of end-user devices and IoT components, which act as the source of data generated by applications and interact with services deployed within this architecture.

The hierarchical structure of the architecture makes it possible to deploy services at any of the three infrastructure levels, depending on their computational requirements and sensitivity to delay. Latency-critical services are placed closer to the edge. However, if a service requires a large amount of computational resources, it would be more appropriate to deploy it at one of the higher levels of the architecture.

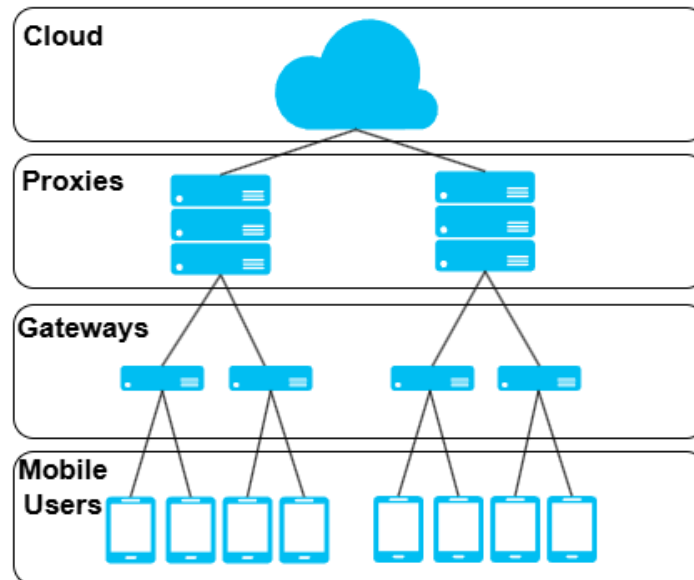


Figure 3.1: MEC Architecture in iFogSim2

3.3 Path-Based Infrastructure Model

In the default iFogSim2 model, there are paths from the Cloud to each individual mobile user. A modified path model has been developed for this project that instead defines paths from the Cloud to each gateway; this represents the sequence of static components that form the route between them.

Although microservices run on a specific FogDevice node, they are also associated with the path on which they operate. Therefore, decisions to migrate a microservice can be made based on the utilisation of the whole path the gateway is associated with, rather than just the gateway itself. For instance, microservices may move from a gateway to a proxy when load becomes too high, meaning the gateway may appear underutilised despite the fact that the path through the gateway is already heavily loaded.

Therefore, by associating microservices with their respective paths, downward migration can be made possible. Upward migration of a microservice is trivial due to a device only ever having a single parent. However, in order to migrate services closer to the edge we must know which services belong to a given path. The path model therefore provides the structure required to dynamically migrate microservices within the infrastructure hierarchy.

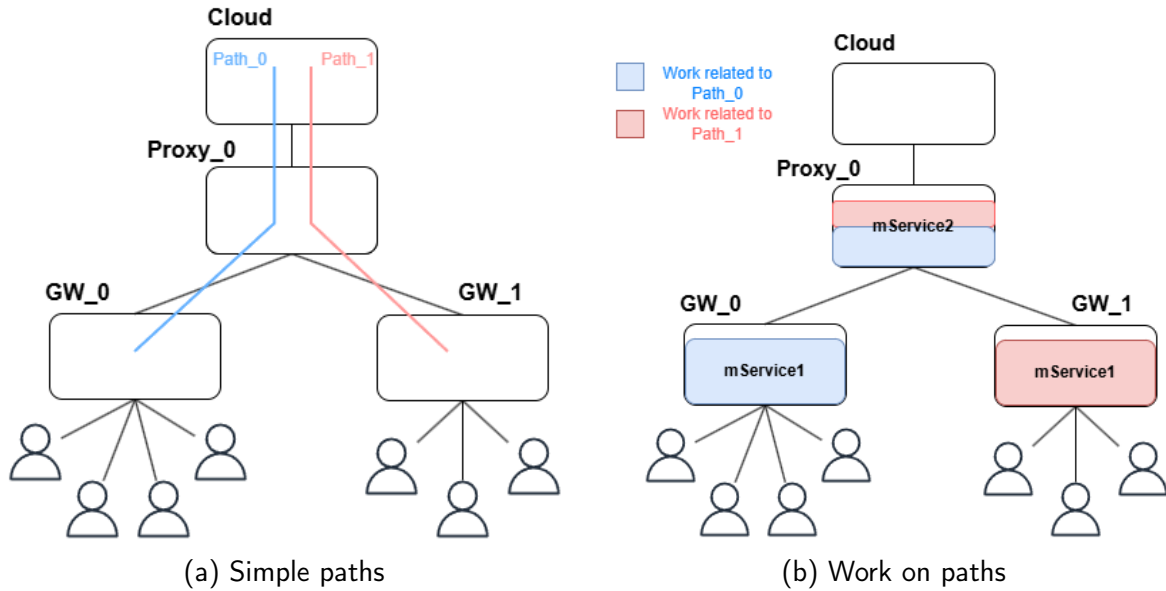


Figure 3.2: Path based architecture

3.4 Limitations of Default iFogSim2 Migration Behaviour

The default migration process in iFogSim2 contains several logical limitations in how microservices are handled when users experience mobility events. In the original migration mechanism, only microservices currently hosted on the user's gateway were considered for migration. If a microservice was migrated higher up the hierarchy (i.e., to a proxy due to high load) at some point previously, it would not be included in subsequent migration decisions. As such, users who were migrating may not have access to the required microservices necessary to execute the application correctly.

In addition, the default mechanism always removes microservices from the previous gateway and recreates them on the destination gateway regardless of whether other users connected to the previous gateway are still using the service. Consequently, microservices may be removed while still required by other users or created regardless of microservice currently existing on the destination gateway.

These issues can result in an incorrect system state where users do not have access to the necessary microservices; therefore, tuples may fail to complete the intended processing loop and prevent the application from executing correctly.

A revised default migration mechanism was developed to address these issues, as defined in Algorithm 1. The updated approach evaluates all microservices used by the application along the user's path. Before deleting a microservice instance, the system checks whether other users still require the service. Similarly, before creating a new instance on the destination path, the system

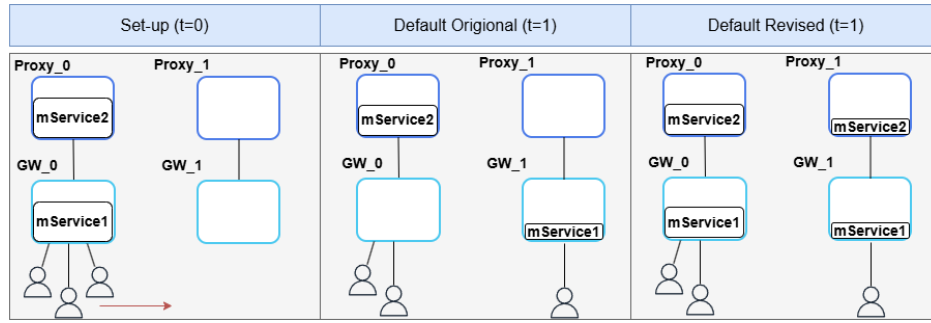


Figure 3.3: Microservice application model

checks whether the microservice already exists there. If not, it is recreated at the same hierarchical level as on the previous path. This maintains consistent service placement, avoids the creation of incorrect modules, and prevents removing modules that other users still use. The revised code implementation can be found in Section 6.1

3.5 Path-Aware Module Placement

The initial placement of microservices is carried out using an extended version of the module placement strategy called ModulePlacementMobileEdgewards. The basic idea behind the placement strategy is that microservices are placed along leaf-to-root paths as defined in the infrastructure hierarchy. For each path, the expected rate of tuples is calculated and this rate is converted into CPU demand for each microservice. Microservices are then placed on the lowest device along the path which provides sufficient CPU capacity for the required workload. In practice, this involves sequentially adding users to the system, creating the appropriate microservices when required and updating the device load. During the initial placement, when a device is already hosting microservices and we attempt to place another user or microservice on the device it results in the load on said device to increase. Should the new workload exceed the threshold for workload, the microservices currently hosted on that device are shifted upwards, to a device with higher capacity in the hierarchy.

HEPA-M enhances this behaviour with a path-aware module placement strategy. In the default implementation, module placement decisions are maintained globally across devices. If a gateway becomes overloaded and a module is forced to shift upwards due to this overload, to the proxy, all paths going through said proxy are required to run the shifted module on that proxy. With the proposed path-aware placement strategy of HEPA-M, placement decisions are stored separately for each path. Therefore, a path-aware placement strategy allows independent instances of the same microservice to be deployed at different gateways along the same path, ensuring that placement decisions made for one gateway path do not unnecessarily affect modules operating on other paths.

This modification is particularly important in the simulated MEC topology where each proxy manages multiple gateways (typically ten gateways per proxy). In the case of the default placement strategy, the forced upward shift from several gateways leads to a significant accumulation of processing load at the proxy level. This accumulated load at the proxy level creates a processing bottleneck at the proxy node. Therefore, the advantage of deploying services close to the edge (i.e., near the gateways) is significantly reduced.

Furthermore, the same placement logic is used during migration events. After a user mobility event, and therefore when a service needs to be evaluated again on a new gateway path, the framework uses the same path-aware placement strategy to decide where modules should be located along

Algorithm 1 Revised Default Microservice Migration Algorithm

Require: Mobility event e

Ensure: Updated parent attachment and revised module placement after mobility

```
1:  $mobile \leftarrow$  mobile device extracted from  $e$ 
2:  $oldParent \leftarrow parentReference[mobile]$ 
3:  $newParent \leftarrow determineParent(mobile)$ 
4: if  $newParent = oldParent$  then
5:   return
6: end if
7:  $update\ parentReference[mobile] \leftarrow newParent$ 
8: set the parent of  $mobile$  to  $newParent$ 
9: remove  $mobile$  from the child list of  $oldParent$ 
10: add  $mobile$  to the child list of  $newParent$ 
11:  $prevParentPath \leftarrow getPathsToCloud(oldParent)$ 
12:  $newParentPath \leftarrow getPathsToCloud(newParent)$ 
13: for all active applications  $app$  on  $mobile$  do
14:    $candidates \leftarrow determineModulesToMigrate(app, prevParentPath)$ 
15:    $prevDeviceMap \leftarrow determineModulesPrevDevice(candidates, prevParentPath)$ 
16:    $newDeviceMap \leftarrow determineModulesNewDevice(candidates, newParentPath, prevDeviceMap)$ 
17:   for all modules  $m$  in  $candidates$  do
18:     if  $prevDeviceMap[m] = newDeviceMap[m]$  then
19:       remove  $m$  from migration consideration
20:     end if
21:   end for
22:    $(toCreate, toRemove) \leftarrow updateModulePlacement(prevDeviceMap, newDeviceMap)$ 
23:   for all modules  $m$  in  $toCreate$  do
24:     if instance count of  $m$  on target device  $> 0$  then
25:       remove  $m$  from  $toCreate$ 
26:     end if
27:   end for
28:   for all modules  $m$  in  $toRemove$  do
29:     if old device will still have another instance of  $m$  after migration then
30:       remove  $m$  from  $toRemove$ 
31:     end if
32:   end for
33:    $updateNewParent(toCreate)$ 
34:    $updateOldParent(toRemove)$ 
35:   for all modules  $m$  in  $toCreate$  do
36:     send MODULE_RECEIVE for  $m$  to its target device
37:   end for
38:   for all modules  $m$  in  $toRemove$  do
39:     locate the running instance of  $m$  on its old device
40:     send MODULE_SEND for  $m$  to its old device
41:   end for
42: end for
```

the new gateway path. If a required microservice is already present on the new gateway path, the microservice is reused; otherwise, a new instance is created at the lowest feasible hierarchical level. By using the path-aware placement logic during migration events and consistently placing services on the lowest feasible device along the path, we improve service placement and minimise bottlenecks. The implemented path logic can be found in Section 6.3.

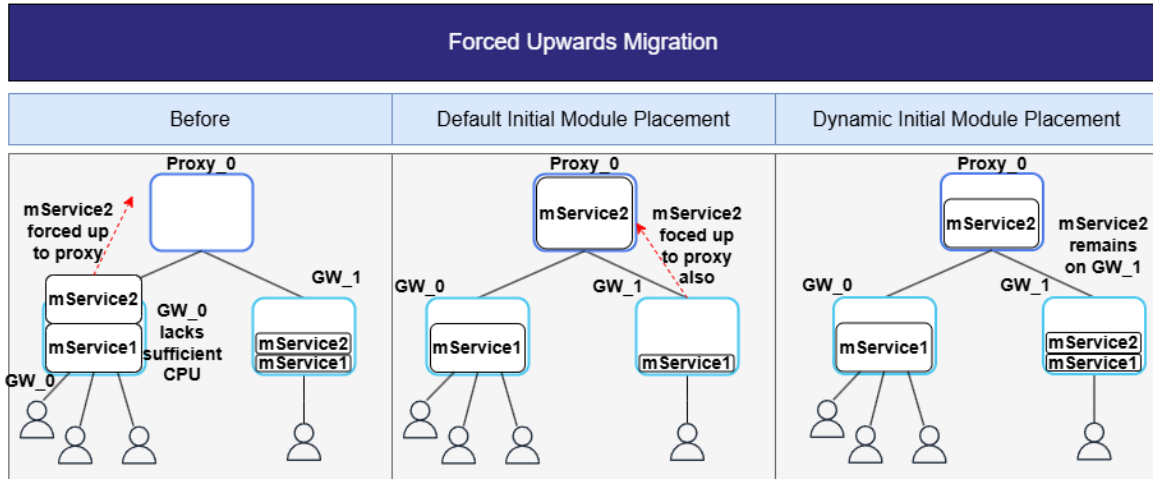


Figure 3.4: Module placement and migration process.

3.6 Resource-Aware Gateway Selection

The default method of selecting the destination gateway for a migrating user in the iFogSim2 framework is based solely on proximity to the user. This is simple to implement, but it does not take into account how heavily utilised the possible destination gateways currently are. As a result, the migration of users to heavily loaded gateways can cause unequal utilisation of resources among gateways, which could also lead to increased processing delays.

A new resource-aware gateway selection strategy has been added to mitigate this limitation. The gateway selection strategy evaluates the CPU utilisation of each possible destination gateway when a user mobility event occurs, and then selects the next best available destination gateway. To assist in evaluating the suitability of a gateway to host additional workloads, a configurable utilisation threshold is applied to determine if the gateway is suitable for additional workload.

If there is an active gateway with utilisation under the defined threshold, the gateway with the least amount of utilisation is selected as the destination. If there is not a single active gateway under the threshold, an inactive gateway may be activated and selected as the destination. In cases where all active gateways meet or exceed the threshold, the gateway with the lowest utilisation is selected as the destination.

The introduction of this gateway selection strategy improves load distribution across the various gateways by limiting the amount of workload received by gateways that are heavily utilised when other available resources exist. Additionally, the configurable utilisation threshold provides the ability to control the trade-off between the two objectives of minimising latency and minimising power usage. Lower threshold values favour improved load balancing and lower latency, while higher thresholds allow greater workload consolidation onto fewer gateways, thereby reducing the number of devices that must remain powered on. The implemented resource-aware algorithm can be found in Section 6.4

Algorithm 2 Default iFogSim2 Mobility Management

```
1: procedure MOBILITYMANAGEMENTDEFAULT( $m, t$ )
2:    $L_m^t \leftarrow$  Location of user  $m$  at time  $t$ 
3:    $p' \leftarrow$  Current parent of  $m$ 
4:    $n \leftarrow$  Operating tier of  $m$ 
5:    $p \leftarrow$  Null
6:    $d_{min} \leftarrow \infty$ 
7:    $F_{n-1} \leftarrow$  Set of fog nodes at level  $n - 1$ 
8:   for  $f_i \in F_{n-1}$  do
9:      $L_{f_i} \leftarrow$  Location of  $f_i$ 
10:     $d \leftarrow$  Distance between  $L_m^t$  and  $L_{f_i}$ 
11:    if  $d < d_{min}$  then
12:       $d_{min} \leftarrow d$ 
13:       $p \leftarrow f_i$ 
14:    end if
15:  end for
16:  if  $p' \neq p$  then
17:    MIGRATEMODULES( $p', p$ )
18:  end if
19: end procedure
```

Algorithm 3 Threshold-Based Resource-Aware Mobility Management

```
1: procedure THRESHOLDBASEDMOBILITYMANAGEMENT( $m, t$ )
2:    $p' \leftarrow$  Current parent of  $m$ 
3:    $b \leftarrow$  Block containing user  $m$ 
4:    $load_M \leftarrow$  Total computational load of migrating modules
5:    $\theta \leftarrow$  utilisation threshold variable
6:    $G_b \leftarrow$  Set of gateway nodes in block  $b$ 
7:   for  $g_i \in G_b$  do
8:      $pathLoad \leftarrow$  Current load on path associated with  $g_i$ 
9:      $utilAfter \leftarrow (pathLoad + load_M) / capacity(g_i)$ 
10:  end for
11:  if active gateway exists where  $utilAfter \leq \theta$  then
12:     $p \leftarrow$  least utilised active gateway
13:  else if gateway exists where  $utilAfter \leq \theta$  then
14:     $p \leftarrow$  least utilised gateway
15:  else
16:     $p \leftarrow$  gateway with lowest utilisation
17:  end if
18:  if  $p' \neq p$  then
19:    MIGRATEMODULES( $p', p$ )
20:  end if
21: end procedure
```

3.7 Dynamic Microservice Migration within the Infrastructure Hierarchy

The proposed solution includes the capability for dynamic service migration within an infrastructure hierarchical structure due to changes in computational load. In the original iFogSim2 model, the placement of microservices remains static along the hierarchical structure, throughout the duration of the simulation run. Therefore, the position of services in the infrastructure does not change as a result of variations in workload demands over time.

HEPA-M provides the ability for microservices to migrate up or down the infrastructure hierarchy path in response to changes in CPU utilisation associated with each microservice. Microservice migrations occur as a result of user mobility events, which cause the framework to evaluate the current resource utilisation across the gateway path for all microservices that exist in the framework.

If the average CPU utilisation associated with the microservices along the gateway path exceeds the specified utilisation threshold for that path, those microservices can be moved upward in the hierarchy to compute nodes with greater CPU capabilities than those existing in the lower layer (e.g., proxy servers). This prevents the gateway node from becoming overloaded and helps preserve the stability of the gateway's processing performance. As demonstrated in Section 4.1.2.

On the other hand, when user migration results in a reduction in the total CPU utilisation associated with the gateway path, those microservices can be migrated downward toward the edge if sufficient resources are available in the lower layer. Thus, services remain closer to the user whenever possible, which results in reduced communication latency, as shown in Simulation 4.1.2.

Through the use of dynamic service migration within the hierarchical structure of the infrastructure, HEPA-M improves the distribution of workload across the system while minimising latency for sensitive applications by allowing them to reside in the lower layers of the system when feasible under the resource constraints of the infrastructure. The implementation can be found in 6.2

3.8 Evaluation Metrics

3.8.1 End-to-End Latency

End-to-End latency is the time it takes for a tuple to complete the full application processing loop from when it is first generated by the sensor to when the processed results reach the final module or actuator. Total time includes network transmission delays, any queuing delays that occur along the path of the tuple, and the amount of time it takes to process each tuple at each fog device along the path of the tuple. In the simulation, the latency of each execution of the application loop is calculated as the difference between when the tuple was sent out (emitted) and when the tuple reaches the end of the application loop.

$$L_{\text{loop}} = t_{\text{receive}} - t_{\text{emit}}$$

where:

- t_{emit} is the time at which the tuple is generated at the beginning of the loop

- t_{receive} is the time at which the tuple reaches the final module or actuator

The average end-to-end latency is calculated across all completed loops during the simulation. Although the average provides a useful representation of the system's overall performance, it can mask outliers. If the system performs poorly for a small minority of cases, this may not be reflected clearly in the average, therefore it is important that the evaluation metrics also capture such variations, particularly for latency-critical applications. We therefore measure worst-case latency to account for outliers.

3.8.2 Energy Consumption

Energy Consumption represents the total amount of energy that the Fog devices use throughout the simulation. The total amount of energy that a Fog Device uses during execution is determined based on the total amount of time that the device was active as well as CPU utilisation rate. In this simulation, we model energy consumption using the PowerModelSquare, in which power consumption increases quadratically with CPU utilisation. As stated in 2.7 we set idle energy consumption to 70% of maximum energy consumption.

$$P(u) = \begin{cases} 0, & u = 0 \\ P_{\text{static}} + (P_{\text{max}} - P_{\text{static}}) \cdot u^2, & 0 < u \leq 1 \end{cases}$$

$$P_{\text{static}} = \alpha \cdot P_{\text{max}}$$

where u is CPU utilisation, P_{max} is maximum power (W), and α represents static power percentage.

The total energy consumption of the system is then calculated by summing the energy consumed by all Fog devices in the infrastructure.

Table 3.1: Power Model Parameters for Fog Devices

Device Type	P_{max} (W)	Static Power (%)	P_{static} (W)
Cloud	1648.00	0.7	1071.20
Proxy Server	107.339	0.7	64.4034
Gateway	107.339	0.7	64.4034
Mobile	87.53	0.7	83.1535

3.8.3 CPU Utilisation

CPU utilisation refers to the computational load placed on a fog node while executing an application. It indicates how much of the CPU resources of a fog node are used to process tuples generated by sensors. In the simulation, each module-to-module application edge defines the computational requirement for processing tuples, referred to as the module edge load. These loads determine the processing demand placed on each fog node.

CPU utilisation is estimated based on the amount of computation required to process a tuple and the rate at which tuples are generated by the sensor. Since the sensor emits tuples periodically, the processing demand over time can be approximated by dividing the module edge computational load by the sensor transmission interval. In the default IFogSim2 a sensor transmission time of 10

seconds was used. In order to increase the workload in this work, the sensor transmission interval was fixed at 7 seconds.

$$U = \frac{L_{module}}{T_{sensor} \cdot MIPS_{total}}$$

where U represents the CPU utilisation, L_{module} is the computational requirement of the module edge, T_{sensor} is the sensor transmission interval, and $MIPS_{total}$ is the total processing capacity of the device.

Chapter 4: Simulation Results and Analysis

In order to assess the functionality and performance of HEPA-M, several simulation experiments were conducted using the iFogSim2 simulator. All experiments were executed on a computer equipped with an AMD Ryzen 7 processor (four cores) and 8GB of RAM. Each simulation terminates once all simulated workload has been processed and no pending events remain in the simulation event queue.

Prior to running the simulations, the physical infrastructure of the network must be defined. Construction of the fog computing topology involves specifying cloud, proxy servers, and gateway nodes which in turn form the hierarchical architecture. In addition to defining the system topology, each Fog device is defined in terms of its processing capability, available memory, available bandwidth for data transmission, and processing characteristics. The values used to describe the capabilities of the Fog devices are derived from previously studied systems in the literature and are summarised in Table 4.1, unless otherwise specified.

Attribute	Cloud	Proxy	Gateway	Mobile
Level	0	1	2	3
CPU (MIPS)	44800	5600	2800	1500
RAM (GB)	40	8	4	2
Uplink Bandwidth (Mbps)	100	10000	10000	10000
Downlink Bandwidth (Mbps)	10000	10000	10000	10000

Table 4.1: Resource configuration of cloud, proxy, gateway and mobile nodes.

In addition to defining the computational capabilities of each Fog device, the location of each device within the network must also be specified. The location of each Fog device is determined using the Melbourne dataset provided with iFogSim2. This dataset contains spatial information describing Fog device locations across the city of Melbourne, Australia, and was developed by the European University Association (EUA). The dataset divides the city into twelve geographic regions, with one proxy server assigned to each region. One or more gateway nodes are associated with the proxy servers for their respective regions. As a result, the traffic patterns generated during the simulation approximate those expected in a real-world urban deployment.



Figure 4.1: Block-wise Fog computing nodes

Data exchanges between microservices in iFogSim2 are modelled using tuples. Each tuple repre-

sents a unit of work that is transmitted from one microservice to another according to the defined application edges. Edges in the application graph are directional and represent tuple flow between microservices.

Each tuple contains two main parameters. The first attribute is the CPU Length, which defines the amount of processing required at the destination microservice to process the tuple. The second attribute is the Network Length, which defines the size of the data being transmitted across the network. Larger Network Length values result in longer transmission times as more data must be transferred between devices.

Table 4.2 below lists the tuple types implemented for the cardiovascular health monitoring application along with the associated CPU and network lengths for each tuple type

Table 4.2: Tuple types and associated CPU and network requirements

Source Module	Destination Module	CPU Length (MI)	Network Length (Bytes)
SENSOR	clientModule	1000	500
clientModule	mService1	2000	500
mService1	mService2	2500	500
mService1	mService3	4000	500
mService2	clientModule	14	500
mService3	clientModule	28	500
clientModule	DISPLAY	14	500
clientModule	DISPLAY	14	500

4.1 Behaviour Simulations

4.1.1 Path-Aware Module Placement

Path-Aware Placement Prevents Unnecessary Upward Migration

The purpose of this test is to demonstrate how the default module placement strategy can incorrectly propagate placement decisions across multiple paths when a single gateway becomes overloaded.

To replicate this scenario, a custom topology was constructed that included a single proxy server linked to two gateway servers. Gateway 0 served three users, and Gateway 1 served one user. As the placement algorithm iterates through users during the initial placement phase, the number of microservice total load on Gateway 0 gradually increases. Eventually, the gateway server reaches the maximum CPU utilisation threshold it can support, and the microservice mService2 must be migrated up to the proxy server.

In Figure 4.2, the expected behaviours of the two placement strategies can be observed. Using the default placement strategy, placement decisions are made at a global level. Once mService2 has been migrated to the proxy as a result of the overload at Gateway 0, the other instance of mService2 on Gateway 1 is also forced to migrate to the proxy despite the gateway containing sufficient computing resources.

The path-aware placement strategy makes placement decisions independently for each gateway path. Thus, once Gateway 0 becomes overloaded, only the instance of mService2 associated with that particular gateway path is migrated to the proxy. The instance of mService2 on Gateway

1 continues to reside on the gateway since there are sufficient computing resources available to support it. As a result, the microservice continues to provide service to the end-user closer to the network edge.

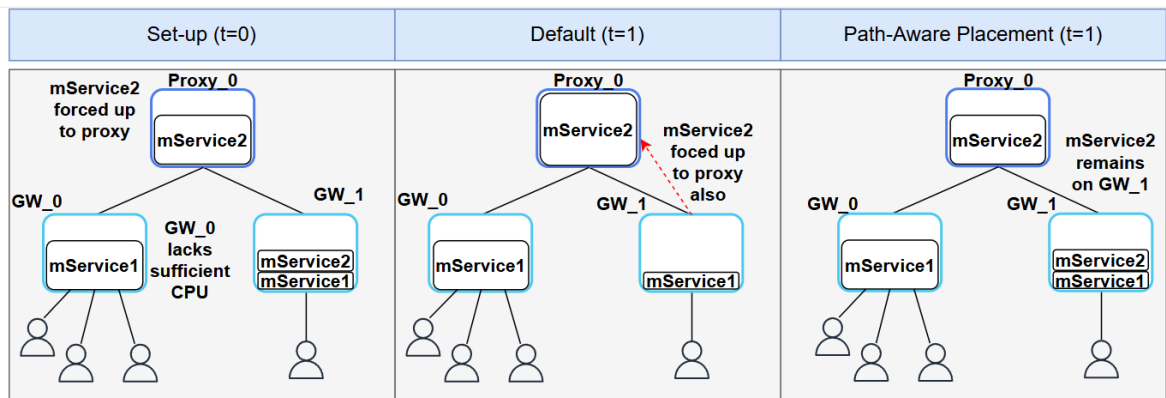


Figure 4.2: Expected framework behaviours when processing the migration.

Results and Discussion

```
Initial Module Placement
=====
proxy-server_0 [mService2] gateway_0 [mService1] gateway_1 [mService1, mService2]
```

The results of the simulation were consistent with the expected behaviour of the two placement strategies. When the default placement strategy is utilised, the increased load on Gateway 0 causes mService2 to migrate to the proxy for both gateway paths. This behaviour occurs because module placement decisions are applied globally across all paths.

When the path-aware placement strategy is employed, only the gateway experiencing the overload migrates its instance of mService2 to the proxy. The second gateway retains its local instance of the microservice, allowing the service to remain closer to the edge. This demonstrates that the proposed path-aware placement strategy prevents unnecessary upward migration of services and reduces the likelihood of bottlenecks forming at higher levels of the system architecture.

Initial Placement Behaviour When All Gateways Are Overloaded

The goal of this test is to show an example case with both gateway paths loaded at the same level, which demonstrates the same location choices by both strategies.

To create such an example case, a simple topology of a single proxy server attached to two gateways was created. Gateway 0 had 3 users and Gateway 1 had 3 users. During the first pass of the placement algorithm processing users to determine the best location for each microservice, it will add the growing number of microservices being added, and increase the total CPU utilisation of the gateways.

When both gateways finally reach their CPU utilisation limits (i.e., they cannot run mService2), mService2 will need to be moved up to the proxy server.

Figure 4.3 shows the behaviour expected of the two placement strategies shown above. Although neither gateway has enough CPU available to host mService2, both the default placement strategy and the path-aware placement strategy move mService2 to the proxy for each gateway path.

Results and Discussion

The simulation demonstrated that both placement strategies behave identically under these con-

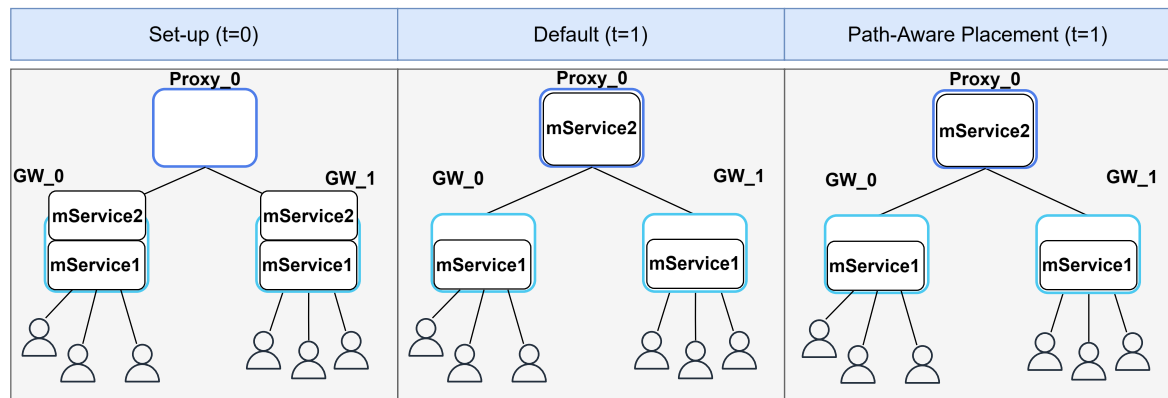


Figure 4.3: Expected framework behaviours when processing the migration.

```
Initial Module Placement
=====
proxy-server_0 [mService2] gateway_0 [mService1] gateway_1 [mService1]
```

ditions. Because both gateways have reached the limit of their available CPU capacity, mService2 needs to be placed at the proxy for both scenarios. It is the intent of this test to illustrate that the path-aware placement strategy is only different than the default placement strategy in those cases where there is remaining capacity on one or more gateway paths to allow mService2 to be hosted on them locally.

Path-Aware Migration from Loaded to Empty Path

In this simulation, a topology was created with two proxy servers, Proxy 0 and Proxy 1, and two gateways, Gateway 0 and Gateway 1, each connected to one of the proxies. As shown in Figure 4.4, when Gateway 0 hosted three users, the microservice mService2 migrated up and was assigned to Proxy 0 because Gateway 0's CPU utilisation exceeded the threshold. At the time the user-mobility event was simulated, Gateway 1 had no users assigned to it.

User-mobility event was simulated by moving a user from Gateway 0 to Gateway 1. The module placement strategy determines which microservices are required to be created on the new path and on which device, along the path they should assigned to. Figure 4.4 illustrates how both placement strategies behave when assigning microservices to the new gateway path.

The default placement strategy places the migrated user's microservices on the proxy server associated with the new gateway path. If no instance of the required microservice exists on the new gateway path, the framework recreates the microservice at the same hierarchical level where it previously existed. Since the previous instance of mService2 was located on a proxy, the framework recreates the service on Proxy 1, regardless of whether the gateway has sufficient available resources.

In contrast, the path-aware placement strategy re-evaluates the available resources along the new gateway path. Because sufficient CPU capacity exists on Gateway 1, the framework assigns the required microservices to the lowest feasible hierarchical level. As a result, the microservice mService2 is placed on Gateway 1 rather than on the proxy.

Results and Discussion

As shown in the simulation results, the placement strategies behave as expected. When using the default placement strategy, mService2 is recreated at Proxy 1 after the migration event. When using the path-aware placement strategy, the microservice mService2 is instead placed on Gateway 1 because sufficient CPU capacity is available. These results demonstrate that HEPA-M keeps

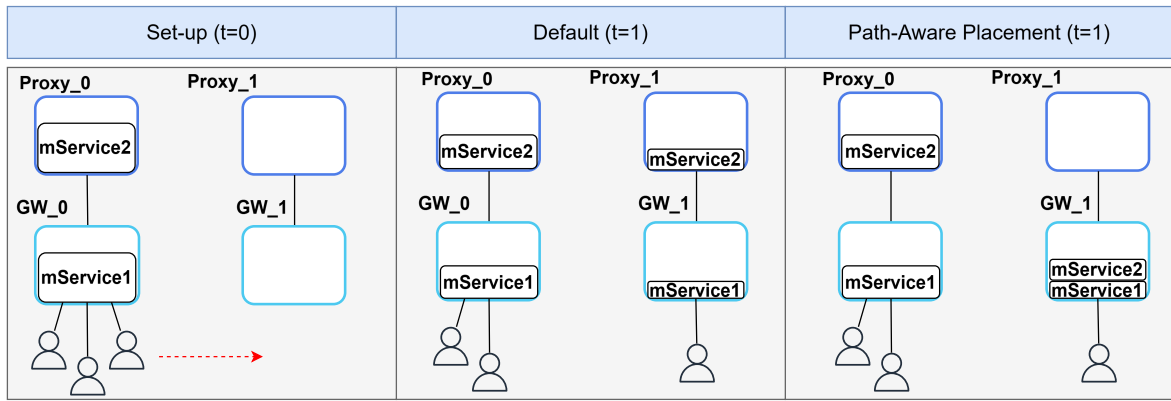


Figure 4.4: Expected framework behaviours when processing the migration.

```
Initial Module Placement
=====
proxy-server_0 [mService2] proxy-server_1 [] gateway_0 [mService1] gateway_1 []

Final Module Placement
=====
proxy-server_0 [mService2] proxy-server_1 [] gateway_0 [mService1] gateway_1 [mService1, mService2]
```

services closer to the edge when possible and avoids unnecessary placement at higher tiers of the infrastructure.

Path-Aware Placement with Uneven Load on the Destination Proxy

To simulate this case study, a network topology with two proxies was created. Proxy 0 was connected to Gateway 0, while Proxy 1 was connected to Gateways 1 and 2. Because Gateway 0 hosted four users, its CPU utilisation exceeded the threshold and the microservice mService2 migrated upward to Proxy 0.

Gateway 2 was also heavily utilised by users, which similarly caused mService2 to migrate to Proxy 1. In contrast, Gateway 1 had no users and therefore no microservices deployed locally.

A user mobility event was then simulated in which a user moved from Gateway 0 to Gateway 1. The placement strategy therefore needed to determine how the required microservices should be deployed along the new gateway path under Proxy 1.

Figure 4.5 illustrates the expected behaviour of both placement strategies. Under the default placement strategy, the migrated user's microservices are placed on the proxy associated with the new gateway path. Because an instance of mService2 already exists on Proxy 1 due to the load from Gateway 2, the migrating user uses this proxy-hosted instance rather than deploying the service on Gateway 1.

In contrast, the path-aware placement strategy evaluates the available resources along the new gateway path. Since Gateway 1 has sufficient CPU capacity, the framework deploys mService2 directly on Gateway 1 instead of using the proxy-hosted instance.

Results and Discussion

The simulation results confirm the expected behaviour. Under the default placement strategy, the migrating user continues to use the instance of mService2 hosted on Proxy 1. Under the path-aware placement strategy, a new instance of mService2 is deployed on Gateway 1 because sufficient resources are available to host the service. This allows the service to remain closer to the edge and avoids unnecessary reliance on higher levels of the infrastructure.

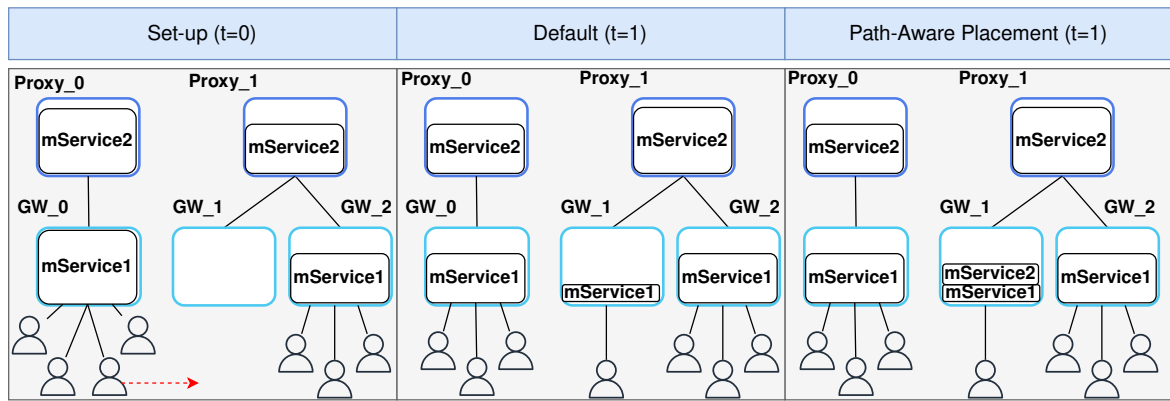


Figure 4.5: Expected framework behaviours when processing the migration.

```

Initial Module Placement
=====
proxy-server_0 [mService2] proxy-server_1 [mService2] gateway_0 [mService1] gateway_1 [] gateway_2 [mService1]

Final Module Placement
=====
proxy-server_0 [mService2] proxy-server_1 [mService2] gateway_0 [mService1] gateway_1 [mService1, mService2] gateway_2 [mService1]

```

Path-Aware Placement with Heavily Loaded Destination Proxy

To test the scenario described above, a network configuration with two proxy servers was created. Proxy 0 had a single gateway, Gateway 0, connected to it, while Proxy 1 had two gateways, Gateway 1 and Gateway 2, connected to it.

At the start of the test, one user session was attached to Gateway 0. Because the gateway reached its CPU utilisation limit, the microservice mService2 migrated to Proxy 0.

Users were also attached to Gateways 1 and 2, causing both gateways to reach their CPU utilisation limits. As a result, the microservice mService2 associated with Gateways 1 and 2 was already hosted on Proxy 1.

Next, a simulated user mobility event was initiated in which one user connected to Gateway 0 moved to Gateway 1. The placement strategy therefore needed to determine which microservices should be provisioned along the new gateway path under Proxy 1.

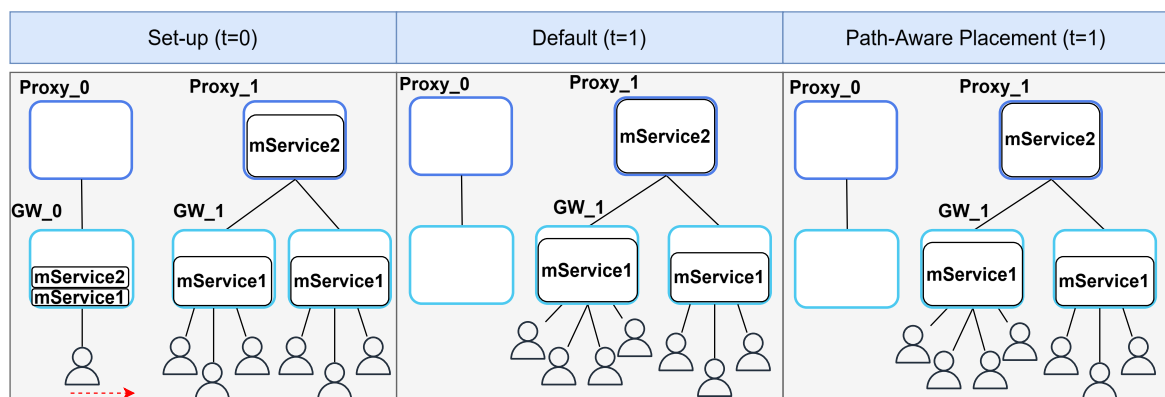


Figure 4.6: Expected framework behaviours when processing the migration.

Figure 4.6 illustrates the expected behaviour for each placement strategy. Since both Gateways 1 and 2 were highly utilised and did not have sufficient CPU capacity to host another microservice, the required instance of mService2 had to be placed at the proxy level.

Results and Discussion

```
Initial Module Placement
=====
proxy-server_0 [] proxy-server_1 [mService2] gateway_0 [mService1, mService2] gateway_1 [mService1] gateway_2 [mService1]
Final Module Placement
=====
proxy-server_0 [] proxy-server_1 [mService2] gateway_0 [] gateway_1 [mService1] gateway_2 [mService1]
```

The simulation results confirm the expected behaviour for both the default placement strategy and the path-aware placement strategy. In both cases, mService2 was placed on Proxy 1 because neither Gateway 1 nor Gateway 2 had sufficient CPU capacity to host the microservice locally. This demonstrates that when no lower-level resources are available, both placement strategies produce the same placement decision.

4.1.2 Hierarchical Microservice Migration

Migration of Microservice up the Hierarchy

To model this scenario, a topology consisting of two proxy servers was created, each connected to a gateway. Proxy 0 was connected to Gateway 0, while Proxy 1 was connected to Gateway 1. At the start of the simulation, Gateway 0 had one user connected and Gateway 1 had two users connected.

A user mobility event was then simulated in which the user connected to Gateway 0 moved to Gateway 1. As a result, Gateway 1 was now hosting three users. Due to the increased workload created by hosting the additional user, the gateway exceeded its CPU utilisation threshold.

At this point, HEPA-M re-evaluated the placement of the microservices along the gateway path. Because Gateway 1 no longer had sufficient computational resources to host mService2, the microservice was migrated upward to Proxy 1 where sufficient resources were available to support the service.

The default implementation of iFogSim2 does not perform this type of hierarchical migration. The initial placement of the microservice on a gateway during the placement phase determines the service location within the framework, and the framework does not relocate the microservice if the gateway later becomes overloaded due to changes in workload caused by user mobility.

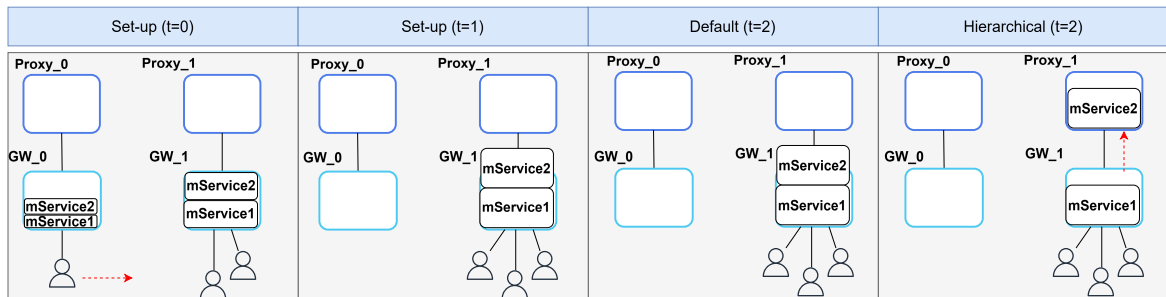


Figure 4.7: Expected framework behaviours when processing the migration.

Results and Discussion

The simulation results confirmed the expected behaviour. After the user mobility event, the increased workload on Gateway 1 caused the gateway to exceed its CPU capacity. HEPA-M migrated the microservice mService2 upward to Proxy 1 to prevent the gateway from becoming overloaded. In contrast, the default implementation of iFogSim2 would retain the microservice on

```

Initial Module Placement
=====
proxy-server_0 [] proxy-server_1 [] gateway_0 [mService1, mService2] gateway_1 [mService1, mService2]

We are migrating modules [mService2] up

Final Module Placement
=====
proxy-server_0 [] proxy-server_1 [mService2] gateway_0 [] gateway_1 [mService1]

```

the gateway, which could lead to unstable performance due to the overloaded device.

Migration of Microservice down the Hierarchy

To replicate this scenario, a network consisting of two proxy servers connected to gateways was created, where Proxy 0 was connected to Gateway 0 and Proxy 1 was connected to Gateway 1. Initially, Gateway 0 hosted three users and therefore exceeded its CPU utilisation threshold, which caused the microservice mService2 to migrate upward and be hosted on Proxy 0.

At the start of the simulation, Gateway 1 had no users connected. A user mobility event was then simulated in which one user moved from Gateway 0 to Gateway 1. As a result, the number of users on Gateway 0 decreased from three to two, reducing the total CPU utilisation of the gateway.

At this point, HEPA-M evaluated the placement of the microservices along the gateway path. Since the CPU utilisation on Gateway 0 had decreased and sufficient capacity was again available, the framework migrated mService2 downward from Proxy 0 back to Gateway 0.

The default implementation of iFogSim2 does not provide this functionality. Once a microservice is placed higher in the hierarchy during the initial placement phase, the framework does not relocate the service to a lower level even if sufficient resources later become available due to changes in workload.

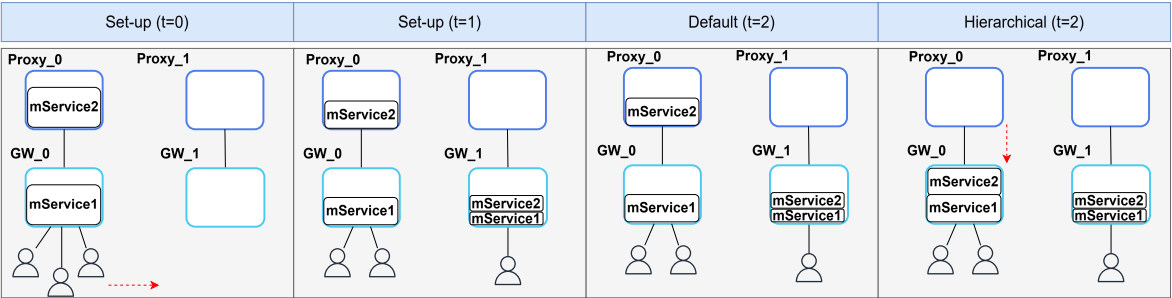


Figure 4.8: Expected framework behaviours when processing the migration.

Results and Discussion

```

Initial Module Placement
=====
proxy-server_0 [mService2] proxy-server_1 [] gateway_0 [mService1] gateway_1 []

Modules we are migrating down [mService2]

Final Module Placement
=====
proxy-server_0 [] proxy-server_1 [] gateway_0 [mService1, mService2] gateway_1 [mService1, mService2]

```

The simulation results confirmed the expected behaviour. When the workload on Gateway 0 was reduced due to the user mobility event, HEPA-M migrated mService2 downward to Gateway 0, allowing the service to again operate closer to the edge. In contrast, the default implementation of iFogSim2 retained mService2 at Proxy 0 even though sufficient resources were available at the gateway. These results demonstrate that the proposed framework dynamically adjusts service placement based on current workload conditions.

No Downward Migration When Gateway Remains Overloaded

To replicate this scenario, a network consisting of two proxy servers connected to gateways was created, where Proxy 0 was connected to Gateway 0 and Proxy 1 was connected to Gateway 1. Initially, Gateway 0 hosted four users, which caused the gateway to exceed its CPU utilisation threshold and resulted in the microservice mService2 being migrated upward to Proxy 0.

A user mobility event was then simulated in which one user moved from Gateway 0 to Gateway 1. As a result, the number of users on Gateway 0 decreased from four to three. However, despite this reduction in workload, Gateway 0 still exceeded the CPU capacity required to host mService2 locally.

At this point, HEPA-M re-evaluated the placement of the microservices along the gateway path. Since Gateway 0 still did not have sufficient computational resources to host mService2, the microservice remained at Proxy 0.

The default implementation of iFogSim2 produces the same behaviour in this case. Because the gateway remains overloaded, the microservice cannot be placed at a lower level in the hierarchy.

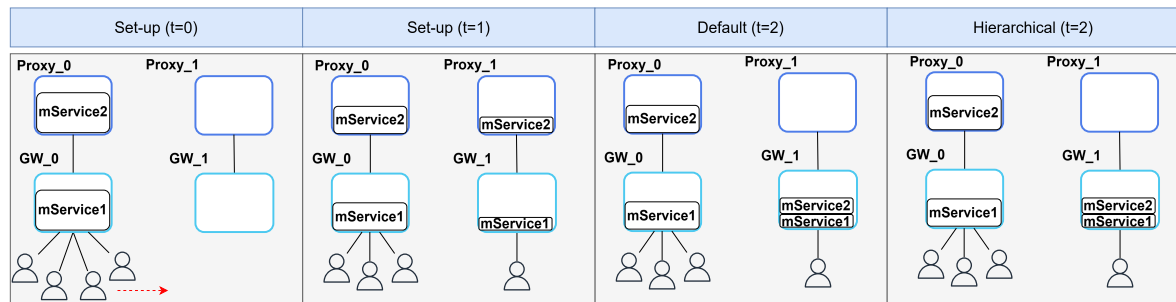


Figure 4.9: Expected framework behaviours when processing the migration.

Results and Discussion

```
Initial Module Placement
=====
proxy-server_0 [mService2] proxy-server_1 [] gateway_0 [mService1] gateway_1 []

Modules we are migrating down []

Final Module Placement
=====
proxy-server_0 [mService2] proxy-server_1 [] gateway_0 [mService1] gateway_1 [mService1, mService2]
```

The simulation results confirm the expected behaviour. After the mobility event, Gateway 0 remained above the CPU utilisation threshold required to host mService2. As a result, both the proposed framework and the default iFogSim2 implementation retained the microservice at Proxy 0. This experiment demonstrates that when the gateway continues to lack sufficient resources, both placement strategies produce the same outcome

4.1.3 Resource-Aware Migration Selection

Resource-Aware Gateway Selection, Selects Active Gateway Under Threshold

To assess the effectiveness of the resource-aware gateway selection strategy, a test network consisting of two proxy servers was established. Proxy Server 0 was connected to a single gateway, while Proxy Server 1 was connected to two gateways. At the beginning of the simulation, a user

was connected to the gateway associated with Proxy Server 0.

A user mobility event was then simulated in which the user moved from the gateway served by Proxy Server 0 into the gateway region managed by Proxy Server 1. Within the region managed by Proxy Server 1, one gateway already hosted a single user while the other gateway had no users connected.

In the default implementation of iFogSim2, the gateway selection strategy simply selects the closest gateway to the migrating user without considering the current resource utilisation of the candidate devices. In contrast, the resource-aware migration strategy evaluates the CPU utilisation of each candidate gateway before selecting the destination device. For this experiment, a relatively high CPU utilisation threshold of 0.85 was specified. Because the gateway already hosting one user remained active and its CPU utilisation was below the threshold, it was selected as the destination gateway for the migrating user.

Figure 4.10 illustrates the expected behaviour of the gateway selection strategy. Although another gateway was available with no load, the framework prioritised the active gateway that remained below the threshold to avoid unnecessarily activating additional infrastructure.

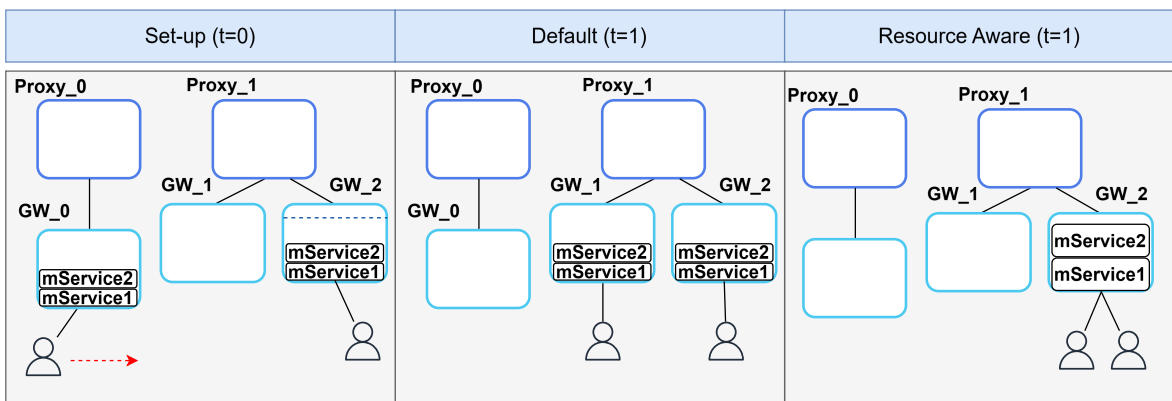


Figure 4.10: Expected framework behaviours when processing the migration.

Results and Discussion

```
Initial Module Placement
=====
proxy-server_0 [] proxy-server_1 [] gateway_0 [mService1, mService2] gateway_1 [] gateway_2 [mService1, mService2]

Chosen ACTIVE gateway (fits): gateway_2
Final Module Placement
=====
proxy-server_0 [] proxy-server_1 [] gateway_0 [] gateway_1 [] gateway_2 [mService1, mService2]
```

The simulation results confirmed the expected behaviour. The migrating user was connected to the gateway that already hosted one user because its CPU utilisation remained below the configured threshold. In contrast, the default gateway selection strategy would simply select the closest gateway to the user without considering its resource utilisation. These results demonstrate how the resource-aware gateway selection strategy prioritises active gateways when sufficient capacity is available, improving resource utilisation and avoiding unnecessary activation of additional gateways.

Resource-Aware Gateway Selection Activates an Idle Gateway

A topology consisting of two proxy servers was used to compare the resource-aware gateway selection approach with the default strategy. Proxy Server 0 was connected to a single gateway, while Proxy Server 1 was connected to two gateways. Initially, a user was connected to the

gateway associated with Proxy Server 0. A user mobility event was then simulated in which the user migrated from the gateway served by Proxy Server 0 to the region managed by Proxy Server 1.

Within this region, Gateway 1 was the closest gateway to the migrating user and was already hosting two users. Although the gateway remained operational, its CPU utilisation exceeded the configured utilisation threshold of 0.85. The second gateway under Proxy Server 1 had no connected users and therefore remained idle.

In the default implementation of iFogSim2, the gateway selection strategy always selects the closest gateway to the user without considering the current resource utilisation. As a result, the default strategy assigned the migrating user to Gateway 1 despite the gateway already exceeding the utilisation threshold.

In contrast, the resource-aware gateway selection strategy evaluates the CPU utilisation of candidate gateways before selecting the destination device. Because Gateway 1 was already operating above the utilisation threshold, it was not selected. Instead, the framework activated the previously unused gateway and assigned the migrating user to that device.

Figure 4.11 illustrates the expected behaviour of the gateway selection strategy. Although Gateway 1 was geographically closer to the user, the framework avoided assigning additional workload to a gateway that had already exceeded the utilisation threshold.

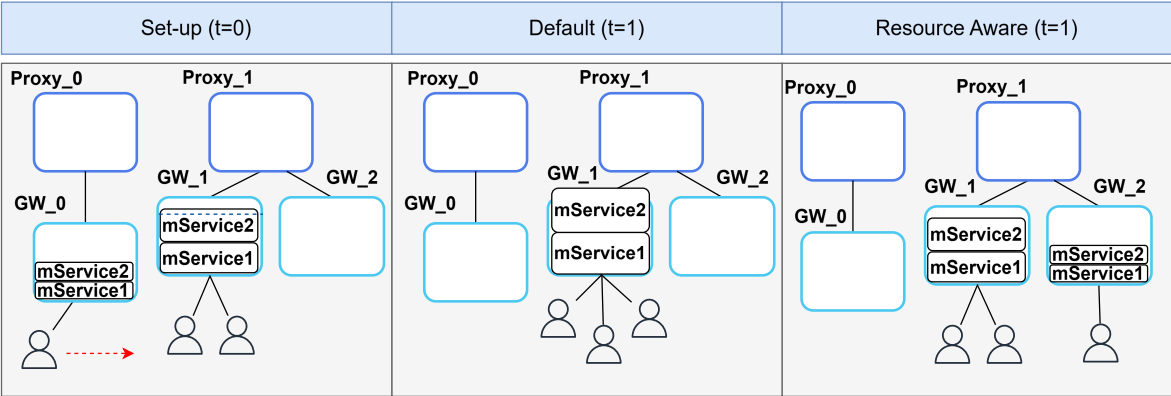


Figure 4.11: Expected framework behaviours when processing the migration.

Results and Discussion

```
Initial Module Placement
=====
proxy-server_0 [] proxy-server_1 [] gateway_0 [mService1, mService2] gateway_1 [mService1, mService2] gateway_2 []

No active gateway fits. Chosen gateway that fits (may be idle): gateway_2

Final Module Placement
=====
proxy-server_0 [] proxy-server_1 [] gateway_0 [] gateway_1 [mService1, mService2] gateway_2 [mService1, mService2]
```

The simulation results confirmed the expected behaviour. Under the default gateway selection strategy, the migrating user was assigned to Gateway 1 because it was the closest device. Under the resource-aware gateway selection strategy, the user was instead assigned to the previously idle gateway because Gateway 1 had already exceeded the utilisation threshold of 0.85. This demonstrates how the proposed strategy can prevent additional load from being assigned to already heavily utilised gateways while still utilising available infrastructure when required.

Resource-Aware Gateway Selection Selects the Least Utilised Gateway

The resource-aware gateway selection strategy is evaluated to observe its behaviour when all candidate gateways for a proxy server exceed the utilisation threshold. A network configuration consisting of two proxy servers, Proxy Server 0 and Proxy Server 1, is used for this evaluation. Proxy Server 0 is connected to a single gateway, while Proxy Server 1 is connected to two gateways. At the beginning of the simulation, a user is connected to the gateway attached to Proxy Server 0.

A user mobility event is then simulated in which the user migrates from the gateway associated with Proxy Server 0 to the region managed by Proxy Server 1. One gateway under Proxy Server 1 hosts one user, while the other gateway hosts two users. The gateway hosting two users is also geographically closer to the migrating user.

For this experiment, the utilisation threshold is set to 0.33. Due to this lower threshold, both gateways under Proxy Server 1 exceed the utilisation limit at the time of migration.

Under the default implementation of iFogSim2, the gateway selection strategy assigns the migrating user to the closest gateway without considering its current utilisation. Consequently, the default strategy selects the closer gateway hosting two users, which causes the gateway to become overloaded due to the additional workload.

In contrast, the resource-aware gateway selection strategy evaluates the utilisation of all candidate gateways before selecting the destination device. Since both gateways exceed the utilisation threshold, the strategy selects the gateway with the lowest utilisation. As a result, the migrating user is assigned to the gateway hosting one user instead of the closer gateway hosting two users.

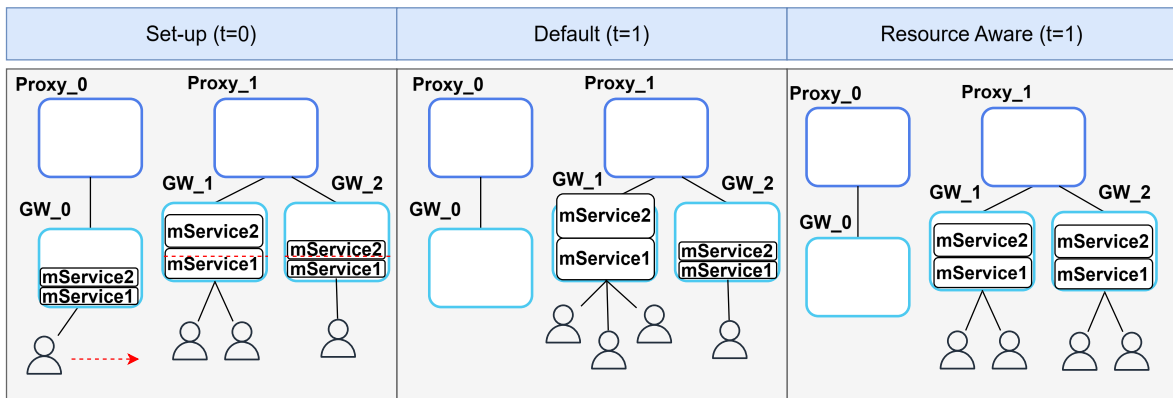


Figure 4.12: Expected algorithm behaviours when processing the module placement.

Results and Analysis

```
Initial Module Placement
=====
proxy-server_0 [] proxy-server_1 [] gateway_0 [mService1, mService2] gateway_1 [mService1, mService2] gateway_2 []

Chosen ACTIVE gateway (fits): gateway_2

Final Module Placement
=====
proxy-server_0 [] proxy-server_1 [] gateway_0 [] gateway_1 [mService1, mService2] gateway_2 [mService1, mService2]
```

The simulation results confirm the expected behaviour. Under the default gateway selection strategy, the migrating user is assigned to the closest gateway, which causes the gateway to become overloaded due to the increased workload. Under the resource-aware gateway selection strategy, the migrating user is assigned to the gateway with the lowest utilisation among the candidate gateways. These results demonstrate that when all gateways exceed the utilisation threshold, the

resource-aware strategy selects the least utilised gateway to better distribute workload across the infrastructure.

Resource-Aware Gateway Selection Selects Least Utilised Path

This final experiment evaluates how a gateway selection strategy that considers available gateway resources behaves when microservices have already migrated upward in the hierarchy, which affects the perceived utilisation of gateway devices. A topology consisting of two proxy servers was used. Proxy Server 0 was connected to a single gateway, while Proxy Server 1 was connected to two gateways. At the beginning of the simulation, a user was connected to the gateway assigned to Proxy Server 0.

A user mobility event was then simulated in which the user migrated from the region served by Proxy Server 0 to the region served by Proxy Server 1. Within this region, one gateway hosted a single user, while the second gateway hosted two users. The gateway hosting two users had previously experienced an overload condition, which caused the microservice mService2 associated with that gateway path to migrate upward to the proxy. As a result, the CPU utilisation reported on that gateway device appeared lower than that of the other gateway.

For this experiment, the utilisation threshold used to determine whether a gateway had exceeded its capacity was set to 0.33. Since both gateways exceeded this threshold when considering the load across their respective paths, both gateways were treated as potential candidates by the resource-aware gateway selection strategy. In this strategy, the utilisation of a gateway is determined based on the total path load associated with that gateway rather than only the load on the gateway device itself. Although the gateway whose microservice had migrated upward appeared to have lower device utilisation, its overall path load remained higher. Consequently, the strategy selected the other gateway as the destination device because it resulted in a better distribution of workload across the infrastructure.

Following the migration event, the additional workload placed on the selected gateway caused it to exceed its capacity, triggering the hierarchical migration mechanism. As a result, the microservice mService2 associated with that gateway path was migrated upward to the proxy level.

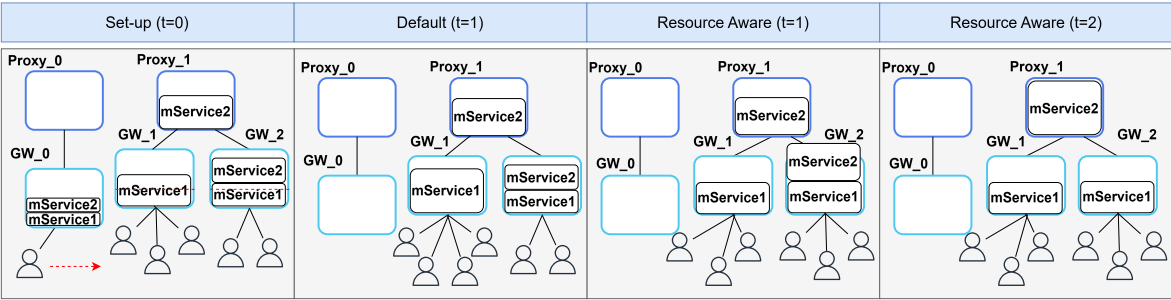


Figure 4.13: Expected framework behaviours when processing the migration.

Results and Analysis

```
Initial Module Placement
=====
proxy-server_0 [] proxy-server_1 [mService2] gateway_0 [mService1, mService2] gateway_1 [mService1] gateway_2 [mService1, mService2]

No gateway fits threshold. Fallback least-busy overall: gateway_2

Final Module Placement
=====
proxy-server_0 [] proxy-server_1 [mService2] gateway_0 [] gateway_1 [mService1] gateway_2 [mService1]
```

The simulation results confirmed the expected behaviour. Although one gateway appeared to have

lower utilisation because mService2 had previously migrated upward to the proxy, the resource-aware strategy evaluated the total path load and selected the gateway that produced a more balanced workload distribution across the infrastructure. After the migration event, the hierarchical migration mechanism moved mService2 to the proxy level. These results demonstrate that combining path-based utilisation measurements with hierarchical service migration leads to a more balanced distribution of computational workload throughout the infrastructure

4.2 Performance Simulations

To evaluate the overall performance of HEPA-M, a series of simulation experiments were conducted using the Melbourne dataset provided by iFogSim2. In these experiments, user mobility was modeled as a random process where users moved between gateway regions at randomly generated times and locations. As a result, each simulation run produced slightly different workload distributions across the infrastructure

Due to the stochastic nature of the mobility model, there may be occasional instances where a simulation run produces extreme conditions. For example, a large number of users may move into the same geographic block, causing gateways within that region to exceed their capacity limits even though other regions may still have available resources. To minimise the impact of these random occurrences, each experiment was executed five times and the average results were analyzed.

Experiments were conducted comparing the performance of the default iFogSim2 implementation and HEPA incorporating all of the enhancements developed in this research. The experiments were also repeated using three different utilisation threshold values (0.33, 0.66, and 1.0) to evaluate how different threshold configurations influence system performance under varying workload conditions.

System performance was measured using two primary metrics: end-to-end latency and total energy consumption. End-to-end latency represents the time required for a tuple to traverse the application processing loop. Total energy consumption represents the total energy used by the fog infrastructure during the simulation.

4.2.1 How Increasing Number of Users Affects System Latency and Total Energy Consumption

The first performance experiment tests how an increase in the number of users will affect the latency of the system as well as the total amount of energy consumed. For this experiment, the number of simulated users was gradually increased from 5 up to 40 users in steps of 5 users. At the conclusion of each simulation run, both end-to-end latency as well as total energy consumption were recorded.

When the system load is low (e.g., when there are only 5 users), the measured latency and energy consumption across all of the different configurations seem to be nearly identical. This occurs since with so few users in the system, it is likely that users are migrating into gateway areas where there is available capacity. Therefore, under these conditions, both the default version of the system and the proposed framework are able to distribute the workload evenly, and the likelihood of gateway congestion is minimal.

However, as the number of users increases in the system, the shortcomings of the default version of the system become more apparent. The default version of the system utilises the closest gateway

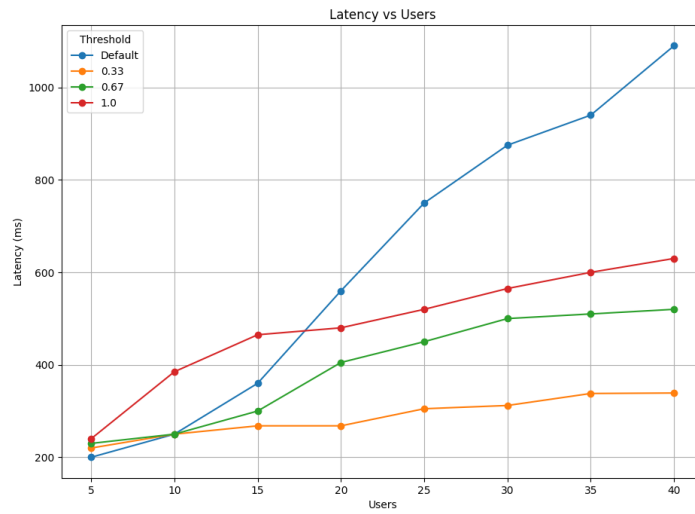


Figure 4.14: Average latency against the number of users under different threshold configurations.

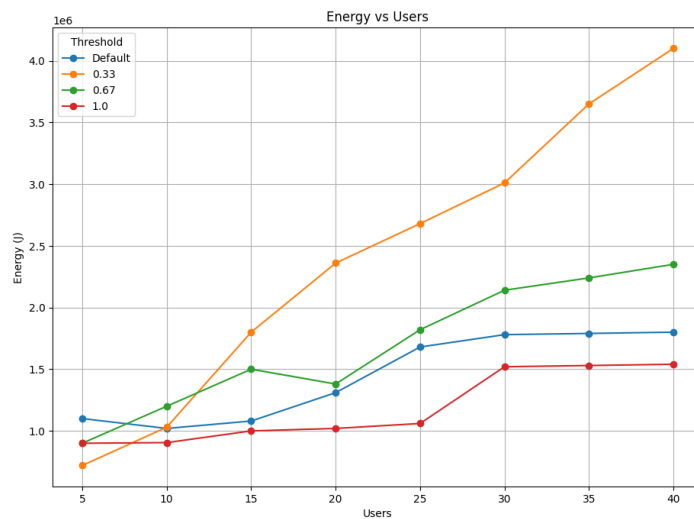


Figure 4.15: Average energy consumption against the number of users under different threshold configurations.

to select a gateway for a given user based solely on proximity to that user and does not consider the degree of utilisation of the selected gateway. As a consequence, some gateways are severely overloaded while others are left under-utilised. The resulting imbalance causes a significant increase in the delay experienced during processing, which is directly reflected in the dramatic rise in measured end-to-end latency seen in the experimental results.

Similar to the end-to-end latency results, the energy consumption results reflect a similar trend. Even though the default" version of the system may have activated fewer fog devices than HEPA-M, the "default" version consumes more energy in total than HEPA-M configuration with a utilisation threshold of 1.00. This is due to two primary factors: the default version of the system has a tendency to cause severe over-load of some gateways while leaving others under-utilised, resulting in high levels of utilisation for extended periods of time. High levels of CPU utilisation cause the energy model to grow non-linearly, thus causing the heavily utilised fog devices to consume disproportionate amounts of energy. Additionally, the processing delays encountered at overloaded gateways extend the duration of task execution and contribute to additional energy consumption.

On the other hand, HEPA-M for distributing workload across available resources is far more effective than the default version of the system. Thresholds with lower values encourage more aggressive distribution of workload across multiple gateways and reduce congestion, resulting in lower latency. Thresholds with higher values allow for consolidation of workload onto fewer devices, reducing the number of active gateways and subsequently reducing energy consumption. HEPA-M configuration using a threshold of 1.00 demonstrated the lowest overall energy usage while maintaining significantly lower latency than the default version of the system.

Overall, the results clearly illustrate the importance of resource-aware gateway selection and hierarchical service migration in maintaining system stability as the workload increases. In addition to reducing latency in systems operating under heavy user loads, HEPA-M also prevents unnecessary energy consumption due to prolonged operation of heavily overloaded devices.

4.2.2 Impact of Gateway Processing Capacity on Latency and Energy Consumption

The second performance experiment investigated the impact of increasing the processing capabilities of gateway devices on system response time (latency) and the total amount of energy consumed by the system. In this experiment, the processing capacity of the gateways was incrementally increased from 1,000 MIPS to 3,000 MIPS in steps of 500 MIPS, while the number of users in the system remained constant at 40 users.

Figure 4.16 illustrates that as the processing capability of the gateways increases, system latency consistently decreases across all configurations of HEPA-M. Increased processing capacity at the gateway level allows more tuples to be processed locally without exceeding the utilisation thresholds. Consequently, the need for microservices to migrate to higher levels in the hierarchy is reduced, which decreases communication delays and improves overall system responsiveness.

Unlike HEPA-M, the default implementation of iFogSim2 exhibits little improvement in latency as gateway capacity increases. This is primarily because the default system selects gateways solely based on proximity and does not consider the current resource utilisation of the gateway. As a result, even when additional computational capacity is available at other gateways in the system, the workload continues to be concentrated on the same set of gateways. Consequently, the architecture is unable to fully utilise the additional computational resources available at the gateway level.

A notable trend can be observed in the energy results at certain gateway processing capacities. At

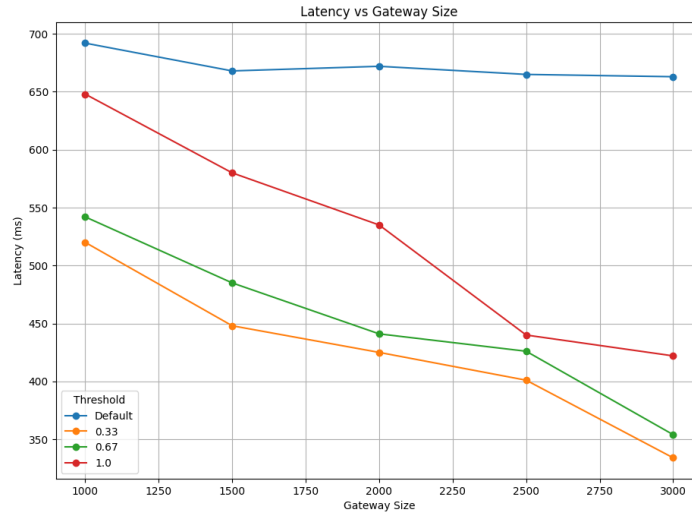


Figure 4.16: Average latency against the gateway capacity under different threshold configurations.

several points along the range of gateway capacities, a significant reduction in energy occurs. Each of these points corresponds to a gateway processing capacity at which mService2, which would normally migrate to the proxy due to insufficient gateway capacity, becomes capable of executing directly on the gateway while remaining below the defined utilisation threshold. When this occurs, the microservice no longer runs on the proxy server, resulting in the proxy being switched off, reducing overall energy consumption whilst simultaneously reducing latency.

The points at which these energy reductions occur depend on the selected utilisation threshold. Higher threshold values allow microservices to remain on gateways with lower processing capacities before migrating to higher levels of the hierarchy. Conversely, lower threshold values require gateways to have greater processing capacity before microservices can remain at the gateway level, since migration is triggered earlier in order to maintain lower utilisation levels. It is this variation in work load allowed to be carried out on a gateway which causes the higher threshold devices to

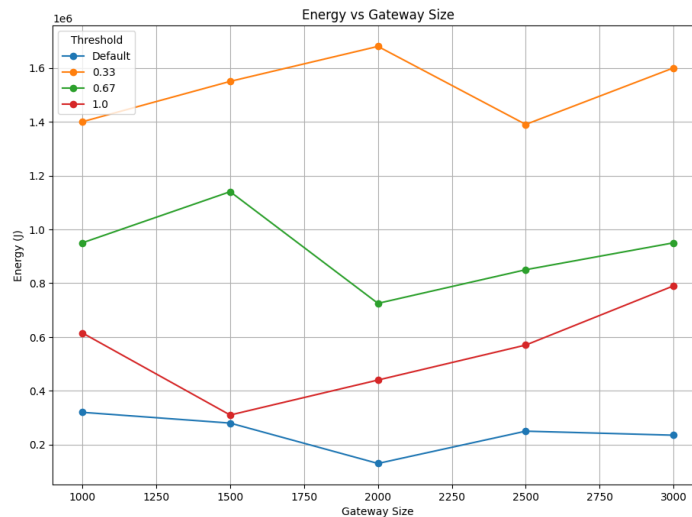


Figure 4.17: Average energy consumption against gateway capacity under different threshold configurations.

Overall, the experimental results demonstrate that HEPA-M is able to effectively utilise increased gateway processing capacity to improve system latency, whereas the default implementation of iFogSim2 fails to fully leverage these additional resources. Furthermore, the results highlight the trade-off between improved performance through increased edge processing capacity and the

corresponding increase in energy consumption within the fog infrastructure.

4.2.3 Impact of Increasing Users within a Single Block

In the third performance experiment, the Melbourne data set's topology was used, with one of the proxies' block chosen for study. Due to the independent operation of each block in the Melbourne network topology, the behaviour that occurs in one block can often counteract the behaviour in another, making certain system behaviours appear either less apparent or "noisy" when looking at the whole system. Therefore, to examine the detailed behaviour of the system in greater detail, the focus of the experiment was placed on a single proxy block with an increasing number of users attached to that block.

In the experiment, the number of users attached to the block was increased incrementally while measuring both the average end-to-end latency and total energy consumption of the system. By doing so, the effects of increasing workload within a localised area of the system could be studied in greater detail.

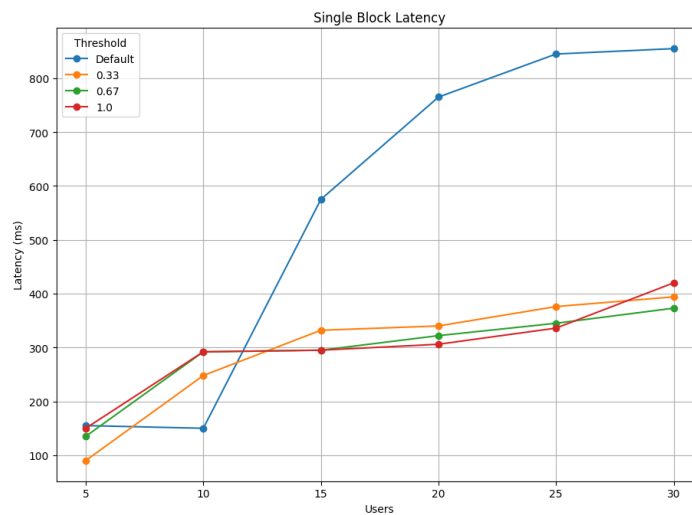


Figure 4.18: Average latency against the number of users under different threshold configurations.

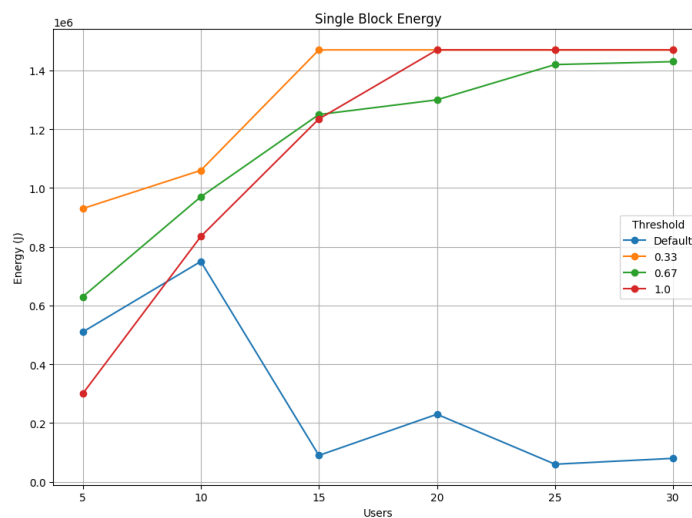


Figure 4.19: Average energy consumption against the number of users under different threshold configurations.

Both the results from the experiment and analysis show that the iFogSim2 default configuration demonstrates the most negative latency performance as the number of users increases. On the contrary, HEPA-M were able to demonstrate much more consistent and predictable latency behaviour.

It is also interesting to note that the three different threshold values (0.33, 0.66, and 1.00) produced very similar latency and energy consumption patterns in the same block. The effect of the threshold value, therefore, is diminished in terms of its overall influence on the system's behaviour when studying only a single block compared to experiments conducted across the entire Melbourne topology.

On the other hand, there were some differences in the energy consumption results. For the 0.33 threshold, energy consumption initially increased slightly and stabilised after approximately 15 users were added to the block. At that time, all available fog devices within the block were operating, and adding additional users would not activate any additional devices within the block, thus maintaining a steady state energy consumption. Both of the 0.66 and 1.00 thresholds exhibited the same behaviour; however, they stabilised at higher numbers of users than the 0.33 threshold did, since the framework was configured to allow the gateways to operate at a higher usage percentage before activating additional devices.

A very interesting pattern developed in the default configuration of the system as a result of this experiment. Because the experiment was limited to a single block, the results of the experiment showed exactly where the spikes in latency occurred and the corresponding drops in energy consumption occurred. Those two events corresponded to the times during module placement when mService2 was required to move up the hierarchy because of gateway overload. In the default configuration, when this migration occurs, all instances of the microservice that are distributed across the gateway paths are moved upward as well, creating a large amount of workload at the proxy level.

Although this reduced the number of active fog devices and thus temporarily decreased energy consumption, it created a huge bottleneck in processing power at the proxy level, which is indicated by a large increase in system latency. Another spike develops in the experiment when mService1 is also forced to move up the hierarchy and creates even more workload at higher levels of the infrastructure.

This type of behaviour is unacceptable due to the production of large and un-predictable changes in how well the system performs. The rapid concentration of services creates severe congestion throughout the infrastructure and creates unstable behaviour in the system's latency. On the other hand, HEPA-M maintains a more even distribution of microservices across the gateway paths, thereby eliminating the possibility of these types of bottlenecks and providing a more stable system performance as the workload increases.

4.2.4 Impact of Increase of Users on Both Average and Tail Latency

The final performance test examined average latency in comparison to tail latency. This experiment aims to evaluate the responsiveness of the entire system and its tail-latency behaviour as the workload increases. We restricted our threshold values to a single 0.33 value as to maintain clarity in graph. We increased the users from 5 to 30 in increments of 5.

These results serve as a confirmation that HEPA-M achieves significantly better latency results for both average and tail. The default framework results in rapid and inconsistent increase in latency, in particular for worst-case. In comparison, HEPA-M shows a slow, gradual increase in latency.

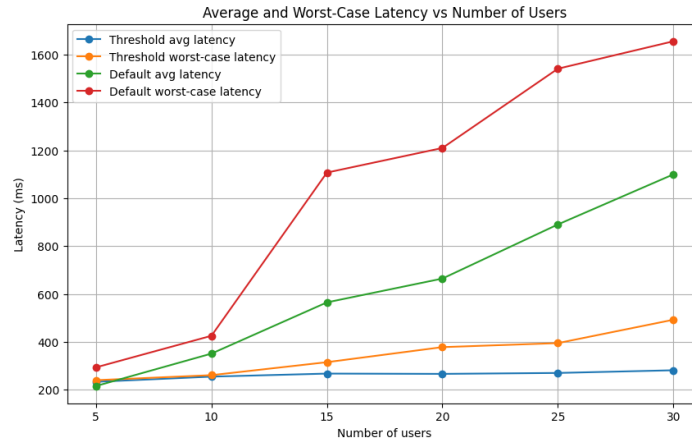


Figure 4.20: Average and worst-case latency against the number of users.

As the number of users increases the performance gap between default and proposed increases as well, suggesting that the proposed framework better distributes work load across all devices in the MEC environment. The performance benefits from HEPA-M are so large that after 10 user the worst-case latency for the proposed is lower than the average of the default. This final experiment reaffirms that HEPA-M provides better latency performance in particular as work load increases.

Chapter 5: Conclusion and Future Work

5.1 Conclusion

In this paper, the capabilities of the existing iFogSim2 framework were extended. During the initial development phase, a critical logical bug was discovered in the existing system that caused microservices to be incorrectly created and destroyed during migration, resulting in application failures. This bug was corrected, enabling correct baseline functionality. HEPA-M builds upon the corrected default with the addition of path-aware module placement, hierarchical microservice migration, a configurable threshold value and resource-aware gateway selection to enhance performance under dynamic user mobility. These extensions enable microservices to respond to changes in workload and be positioned more effectively throughout the infrastructure, whilst balancing latency and energy consumption.

Simulation results indicate that HEPA-M provides significant improvements over the corrected baseline in terms of latency, energy efficiency, and stability. The baseline method relies on proximity-based decisions, often resulting in overloaded gateways, inefficient resource usage, and sharp increases in latency. In contrast, HEPA-M distributes workload more efficiently, reduces bottlenecks, prevents sharp increases in latency and maintains services closer to the edge when possible.

In summary, the results support that resource awareness, path awareness, and hierarchical migration are key components for effective operation in MEC systems, particularly for latency-sensitive applications.

5.2 Future Work

There are several directions that this study could be extended. Firstly, this study has assumed one application model. Heterogeneous application models of varying latency and resources could be investigated, better reflecting real world MEC environment.

Secondly, the mobility model used in this study represents movement randomly. The use of realistic mobility patterns and/or trace-based mobility models could lead to additional studies which assess the true behaviour of systems in a real world MEC environment.

Finally, an additional issue identified but outside the scope of this project relates to tuple routing during migration. In the current system, as tuples traverse upwards they record which devices on the path they were processed on, and during downward migration back to the user they are broadcast to all child devices, and only the matching device from the along path processes the tuple. This approach has two issues, firstly it is inefficient as all child devices must determine if they are required to process tuple and secondly, during migration a user may end up on a new path, meaning the tuple is routed to the incorrect devices. Leaving the user without ever receiving the tuple. A more targeted routing mechanism could significantly improve both efficiency and correctness, representing a valuable direction for further research.

Bibliography

1. Mell, P. & Grance, T. *The NIST Definition of Cloud Computing* tech. rep. (NIST, 2011).
2. Et al., M. A. *Above the Clouds: A Berkeley View of Cloud Computing* tech. rep. (UC Berkeley, 2009).
3. Satyanarayanan, M. The Emergence of Edge Computing. *Computer* **50**, 30–39 (2017).
4. Shi, W., Cao, J., Zhang, Q., Li, Y. & Xu, L. Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal* **3**, 637–646 (2016).
5. ETSI. *Mobile Edge Computing – Introductory Technical White Paper* tech. rep. (2014).
6. Atzori, L., Iera, A. & Morabito, G. The Internet of Things: A Survey. *Computer Networks* **54**, 2787–2805 (2010).
7. Gubbi, J., Buyya, R., Marusic, S. & Palaniswami, M. Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions. *Future Generation Computer Systems* **29**, 1645–1660 (2013).
8. Mach, P. & Becvar, Z. Mobile Edge Computing: A Survey on Architecture and Computation Offloading. *IEEE Communications Surveys & Tutorials* (2020).
9. Varghese, B. & Buyya, R. Next Generation Cloud Computing: New Trends and Research Directions. *Future Generation Computer Systems* (2016).
10. Mao, Y., You, C., Zhang, J., Huang, K. & Letaief, K. B. A Survey on Mobile Edge Computing: The Communication Perspective. *IEEE Communications Surveys & Tutorials* (2020).
11. Mao, Y., You, C., Zhang, J., Huang, K. & Letaief, K. B. A Survey on Mobile Edge Computing: The Communication Perspective. *IEEE Communications Surveys & Tutorials* (2017).
12. ITU-T. *Definition of Quality of Experience (QoE)* tech. rep. (2008).
13. Wu, W., Tan, Z., Li, R. & Huang, J. QoE-based Resource Allocation for Mobile Edge Computing. *IEEE Access* (2018).
14. Xu, X. et al. BeCome: Blockchain-Enabled Computation Offloading for IoT in Mobile Edge Computing. *IEEE Transactions on Industrial Informatics* (2018).
15. Puliafito, C., Mingozzi, E., Longo, S., Puliafito, F. & Rana, O. Fog Computing for the Internet of Things: A Survey. *ACM Transactions on Internet Technology* (2019).
16. Li, Y., Wang, W., Jin, D. & Chen, S. Optimal Mobile Service Migration in Mobile Edge Computing. *IEEE Communications Letters* (2018).
17. Et al., T. T. On Multi-access Edge Computing: A Survey of the Emerging 5G Network Edge Architecture. *IEEE Communications Surveys & Tutorials* (2017).
18. Guo, Y., Zhang, S. & Wang, S. Energy-Efficient Dynamic Offloading and Resource Scheduling in Mobile Edge Computing. *IEEE Transactions on Wireless Communications* (2019).
19. Chen, X., Jiao, L., Li, W. & Fu, X. Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing. *IEEE/ACM Transactions on Networking* (2020).
20. Wang, S. et al. Dynamic Service Placement for Mobile Micro-Clouds with Predicted Future Costs. *IEEE Transactions on Parallel and Distributed Systems* (2018).
21. Khoshsirat, A., Perin, G. & Rossi, M. Divide and Save: Splitting Workload Among Containers in an Edge Device to Save Energy and Time. *arXiv preprint arXiv:2302.06478* (2023).
22. Jiang, C. et al. Energy-aware edge computing: A survey. *Computer Communications* (2020).

-
23. Zaman, S. K. *et al.* COME-UP: Computation Offloading in Mobile Edge Computing. *Applied Sciences* (2022).
 24. Beloglazov, A. & Buyya, R. Energy Efficient Resource Management in Virtualized Cloud Data Centers. *Future Generation Computer Systems* (2012).
 25. Rizvandi, N. B. *et al.* Some Observations on Optimal Frequency Selection in DVFS. *IEEE/ACM (distributed computing research)* (2012).
 26. Calheiros, R. N., Ranjan, R., Beloglazov, A., Rose, C. D. & Buyya, R. CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments. *Software: Practice and Experience* (2011).
 27. Mahmud, M. M. H. & *et al.*, R. B. iFogSim2: An Extended Simulator for Edge and Fog Computing Environments. *Journal of Systems and Software* (2022).

Chapter 6: Appendix

Gitlab Repo: <https://csgitlab.ucd.ie/jamesfinn/4yp>

6.1 Revised Default Mobility Controller Code

```
1  protected void migrateModules(FogDevice fogDevice, FogDevice prevParent, FogDevice
   ↪  newParent) {
2      parentReference.put(fogDevice.getId(), newParent.getId());
3      List<String> migratingModules = new ArrayList<>();
4      if (prevParent.getId() != newParent.getId()) {
5
6          // common ancestor policy
7          List<Integer> newParentPath = getPathsToCloud(newParent.getId());
8          List<Integer> prevParentPath = getPathsToCloud(prevParent.getId());
9          int commonAncestor = determineAncestor(newParentPath, prevParentPath);
10
11         fogDevice.setParentId(newParent.getId());
12         newParent.getChildToLatencyMap().put(fogDevice.getId(),
   ↪  fogDevice.getUplinkLatency());
13         newParent.addChild(fogDevice.getId());
14         prevParent.removeChild(fogDevice.getId());
15         newParent.getChildToLatencyMap().put(fogDevice.getId(),
   ↪  fogDevice.getUplinkLatency());
16
17         for (String applicationName : fogDevice.getActiveApplications()) {
18             ModulePlacement modulePlacement =
   ↪  getAppModulePlacementPolicy().get(applicationName);
19             migratingModules = determineModulesToMigrate(modulePlacement,
   ↪  prevParentPath, newParentPath);
20
21             Map<String, Integer> migratingModulesPrevDeviceMap =
22                 determineModulesPrevDevice(modulePlacement, migratingModules,
   ↪  prevParentPath);
23             Map<String, Integer> migratingModulesNewDeviceMap =
24                 determineModulesNewDevice(modulePlacement, migratingModules,
   ↪  newParentPath,
25                 migratingModulesPrevDeviceMap);
26
27             Application application = getApplications().get(applicationName);
28
29             removeModulesWithSameParent(migratingModules,
   ↪  migratingModulesPrevDeviceMap,
```

```

30         migratingModulesNewDeviceMap);
31     updateModulePlacement(application, migratingModulesPrevDeviceMap,
32         migratingModulesNewDeviceMap, commonAncestor, prevParentPath,
33         ↪ newParentPath);
34     updateUserPathway(applicationName, fogDevice, prevParent, newParent);
35
36     handleMigrateModulesOnPath(applicationName, prevParentPath,
37         ↪ newParentPath);
38 }
39
40 // hook for dynamicMobilityController
41 protected void handleMigrateModulesOnPath(String applicationName, List<Integer>
42     ↪ prevPath,
43     List<Integer> newPath) {
44 }
45
46 private List<String> determineModulesToMigrate(ModulePlacement modulePlacement,
47     List<Integer> prevParentPath, List<Integer> newParentPath) {
48
49     Set<String> modulesToMigrate = new LinkedHashSet<>();
50
51     // we can't migrate modules on client or fixed ie mservice 3
52     for (int i = 0; i < prevParentPath.size(); i++) {
53         if (getFogDeviceById(prevParentPath.get(i)).getLevel() > 0
54             && getFogDeviceById(prevParentPath.get(i)).getLevel() < 3) {
55             modulesToMigrate.addAll(modulePlacement.getModulesOnDevice()
56                 .get(prevParentPath.get(i)));
57         }
58     }
59     return new ArrayList<>(modulesToMigrate);
60 }
61
62 private Map<String, Integer> determineModulesPrevDevice(ModulePlacement
63     ↪ modulePlacement,
64     List<String> modulesToMigrate, List<Integer> prevPath) {
65     HashMap<String, Integer> moduleToDeviceMap = new HashMap<>();
66     HashSet<String> moduleSet = new HashSet<>(modulesToMigrate);
67
68     for (int deviceId : prevPath) {
69         for (String moduleName : modulePlacement.getModulesOnDevice().get(deviceId))
70             ↪ {
71                 if (moduleSet.contains(moduleName) &&
72                     ↪ !moduleToDeviceMap.containsKey(moduleName)) {
73                     moduleToDeviceMap.put(moduleName, deviceId);
74                 }
75             }
76     }
77 }

```

```

75     return moduleToDeviceMap;
76 }
77
78 protected Map<String, Integer> determineModulesNewDevice(ModulePlacement
    ↪ modulePlacement,
79     List<String> modulesToMigrate, List<Integer> newPath,
80     Map<String, Integer> prevModuleDeviceIds) {
81     HashMap<String, Integer> moduleToDeviceMap = new HashMap<>();
82     HashSet<String> moduleSet = new HashSet<>(modulesToMigrate);
83
84     for (int deviceId : newPath) {
85         for (String moduleName : modulePlacement.getModulesOnDevice().get(deviceId))
            ↪ {
86             if (moduleSet.contains(moduleName)) {
87                 moduleToDeviceMap.put(moduleName, deviceId);
88                 moduleSet.remove(moduleName);
89             }
90         }
91     }
92
93     for (String moduleName : moduleSet) {
94         int deviceId = prevModuleDeviceIds.get(moduleName);
95         int level = getFogDeviceById(deviceId).getLevel();
96         moduleToDeviceMap.put(moduleName, newPath.get(newPath.size() - 1 - level));
97     }
98
99     return moduleToDeviceMap;
100 }
101
102 private void removeModulesWithSameParent(List<String> migratingModule,
103     Map<String, Integer> migratingModulesPrevDevice,
104     Map<String, Integer> migratingModulesNewDevice) {
105     List<String> modulesToRemove = new ArrayList<>();
106
107     for (String moduleName : migratingModule) {
108         if (migratingModulesNewDevice.get(moduleName)
109             .equals(migratingModulesPrevDevice.get(moduleName))) {
110             migratingModulesNewDevice.remove(moduleName);
111             migratingModulesPrevDevice.remove(moduleName);
112             modulesToRemove.add(moduleName);
113         }
114     }
115     for (String moduleName : modulesToRemove) {
116         migratingModule.remove(moduleName);
117     }
118 }
119
120 // todo this logic belongs in module placement
121 protected void updateModulePlacement(Application application,
122     Map<String, Integer> migratingModulesPrevDevice,
123     Map<String, Integer> migratingModulesNewDevice, int commonAncestor,

```

```

124         List<Integer> oldParentPath, List<Integer> newParentPath) {
125
126         ModulePlacementMobileEdgewards placement = getModulePlacementPolicy();
127         List<String> modulesToCreate =
128             determineModulesToCreateOnNewDevice(migratingModulesNewDevice, placement, 1,
129                 ↪ newParentPath);
130         creatModulesOnDevices(modulesToCreate, migratingModulesNewDevice, application,
131             commonAncestor, true);
132         updateNewParent(migratingModulesNewDevice, placement, 1, newParentPath, false);
133
134         List<String> modulesToRemove =
135             determineModulesToRemoveOldParent(migratingModulesPrevDevice, placement, 1,
136                 ↪ oldParentPath);
137         removeModuleFromDevices(modulesToRemove, migratingModulesPrevDevice, placement,
138             application, commonAncestor, true);
139         updateOldParent(migratingModulesPrevDevice, placement, 1, oldParentPath, false);
140     }
141
142     protected void updateOldParent(Map<String, Integer> migratingModules,
143         ModulePlacementMobileEdgewards modulePlacement, int numMigrating,
144         List<Integer> oldParentPath, boolean isModuleMigration) {
145
146         for (String moduleName : migratingModules.keySet()) {
147             if ("mService3".equals(moduleName)) continue; // pin to cloud
148             int id = migratingModules.get(moduleName);
149
150             // module mips load of single user
151             double moduleLoad = modulePlacement.getModuleLoad().get(moduleName) *
152                 ↪ numMigrating;
153
154             Map<String, Double> map = modulePlacement.getCurrentModuleLoadMap().get(id);
155
156             double totalModuleLoad = map.getOrDefault(moduleName, 0.0);
157             totalModuleLoad -= moduleLoad;
158             modulePlacement.getCurrentModuleLoadMap().get(id).put(moduleName,
159                 ↪ totalModuleLoad);
160
161             double cpuLoad = modulePlacement.getCurrentCpuLoad().getOrDefault(id, 0.0);
162             cpuLoad -= moduleLoad;
163             if (Math.abs(cpuLoad) < 0.005) cpuLoad = 0.0;
164             modulePlacement.getCurrentCpuLoad().put(id, cpuLoad);
165
166             Map<String, Integer> map1 =
167                 ↪ modulePlacement.getCurrentModuleInstanceNum().get(id);
168             int numInstances = -1;
169             numInstances = map1.getOrDefault(moduleName, 0);
170             numInstances -= numMigrating;
171             modulePlacement.getCurrentModuleInstanceNum().get(id).put(moduleName,
172                 ↪ numInstances);
173
174             if (numInstances <= 0) {

```

```

169         modulePlacement.getCurrentModuleMap().get(id).remove(moduleName);
170     }
171 }
172 }
173
174 protected List<String> determineModulesToRemoveOldParent(
175     Map<String, Integer> migratingModules,
176     ModulePlacementMobileEdgewards modulePlacement, int numMigrating,
177     List<Integer> oldParentPath) {
178     List<String> modulesToRemove = new ArrayList<>();
179
180     for (String moduleName : migratingModules.keySet()) {
181         int id = migratingModules.get(moduleName);
182
183         Map<String, Integer> map =
184             ↪ modulePlacement.getCurrentModuleInstanceNum().get(id);
185         int numInstances = map.getOrDefault(moduleName, 0);
186         if (numInstances <= numMigrating) modulesToRemove.add(moduleName);
187     }
188     return modulesToRemove;
189 }
190
191 protected void updateNewParent(Map<String, Integer> migratingModules,
192     ModulePlacementMobileEdgewards modulePlacement, int numMigrating,
193     List<Integer> newParentPath, boolean isModuleMigration) {
194
195     for (String moduleName : migratingModules.keySet()) {
196         if ("mService3".equals(moduleName)) continue; // pin to cloud
197         int id = migratingModules.get(moduleName);
198
199         // module mips load of single user
200         double moduleLoad = modulePlacement.getModuleLoad().get(moduleName) *
201             ↪ numMigrating;
202
203         double totalModuleLoad = modulePlacement.getCurrentModuleLoadMap().get(id)
204             .getOrDefault(moduleName, 0.0);
205         totalModuleLoad += moduleLoad;
206         modulePlacement.getCurrentModuleLoadMap().get(id).put(moduleName,
207             ↪ totalModuleLoad);
208
209         double cpuLoad = modulePlacement.getCurrentCpuLoad().getOrDefault(id, 0.0);
210         cpuLoad += moduleLoad;
211         modulePlacement.getCurrentCpuLoad().put(id, cpuLoad);
212
213         int numInstances = -1;
214         numInstances = modulePlacement.getCurrentModuleInstanceNum().get(id)
215             .getOrDefault(moduleName, 0);
216         numInstances += numMigrating;
217         modulePlacement.getCurrentModuleInstanceNum().get(id).put(moduleName,
218             ↪ numInstances);
219     }
220 }

```

```

216         if (numInstances == numMigrating) {
217             List<String> mods = modulePlacement.getCurrentModuleMap().get(id);
218             if (mods != null && !mods.contains(moduleName))
219                 mods.add(moduleName);
220         }
221     }
222 }
223
224 protected List<String> determineModulesToCreateOnNewDevice(
225     Map<String, Integer> migratingModules,
226     ModulePlacementMobileEdgewards modulePlacement,
227     int numMigrating,
228     List<Integer> newParentPath) {
229     List<String> modulesToCreate = new ArrayList<>();
230     for (String moduleName : migratingModules.keySet()) {
231         int id = migratingModules.get(moduleName);
232
233         if (modulePlacement.getCurrentModuleInstanceNum().get(id) == null) continue;
234         int numInstances = modulePlacement.getCurrentModuleInstanceNum().get(id)
235             .getOrDefault(moduleName, 0);
236
237         if (numInstances == 0) {
238             modulesToCreate.add(moduleName);
239         }
240     }
241
242     return modulesToCreate;
243 }
244
245 protected void removeModuleFromDevices(List<String> modules,
246     Map<String, Integer> migratingModules,
247     ModulePlacementMobileEdgewards modulePlacement,
248     Application application, int commonAncestor, boolean hasDelay) {
249
250     for (String moduleName : modules) {
251         int id = migratingModules.get(moduleName);
252
253         double upDelay = 0.0;
254         try {
255             if (hasDelay)
256                 upDelay = getUpDelay(id, commonAncestor,
257                     ↪ application.getModuleByName(moduleName));
258             } catch (Exception e) {
259                 upDelay = 0.0;
260             }
261
262         removeModule(id, moduleName, application, upDelay);
263     }
264 }

```

```

265 protected void removeModule(int id, String moduleName, Application application,
    ↪ double delay) {
266     AppModule running = getRunningModuleOnDevice(id, moduleName);
267     if (running == null) {
268         System.out.println("Skip destroy: no running instance of " + moduleName
269             + " on device " + id);
270         return;
271     }
272
273     JSONObject jsonSend = new JSONObject();
274     jsonSend.put("module", running);
275     jsonSend.put("delay", delay);
276     jsonSend.put("application", application);
277
278     System.out.println("Removing Module " + moduleName + " deivce id " + id
279         + " delay is " + delay);
280     send(id, 0.0, FogEvents.MODULE_SEND, jsonSend);
281 }
282
283 /**
284  * Finds the actual running AppModule instance by name on a device.
285  */
286 protected AppModule getRunningModuleOnDevice(final int deviceId, final String
    ↪ moduleName) {
287     FogDevice dev = getFogDeviceById(deviceId);
288     if (dev == null) return null;
289     for (Vm vm : dev.getHost().getVmList()) {
290         if (vm instanceof AppModule && ((AppModule)
    ↪ vm).getName().equals(moduleName)) {
291             return (AppModule) vm;
292         }
293     }
294     return null;
295 }
296
297 protected void creatModulesOnDevices(List<String> modules,
298     Map<String, Integer> migratingModules, Application application,
299     int commonAncestor, boolean hasDelay) {
300
301     for (String moduleName : modules) {
302         int id = migratingModules.get(moduleName);
303         double downDelay = 0.0;
304         try {
305             if (hasDelay) getDownDelay(id, commonAncestor,
    ↪ application.getModuleByName(moduleName));
306         } catch (Exception exception) {
307             downDelay = 0.0;
308         }
309         createModule(id, moduleName, application, downDelay);
310     }
311 }

```

```

312
313 protected void createModule(int id, String moduleName, Application application,
    ↳ double delay) {
314     JSONObject jsonReceive = new JSONObject();
315     jsonReceive.put("module", application.getModuleByName(moduleName));
316     jsonReceive.put("delay", delay);
317     jsonReceive.put("application", application);
318
319     System.out.println("Creating Module " + moduleName + " deivce id " + id);
320     send(id, 0.0, FogEvents.MODULE_RECEIVE, jsonReceive);
321 }
322
323 private void updateUserPathway(String applicationName, FogDevice fogDevice,
324     FogDevice prevParent, FogDevice newParent) {
325     ModulePlacement modulePlacement =
326     ↳ getAppModulePlacementPolicy().get(applicationName);
327
328     ↳ modulePlacement.getModulesOnPath().get(fogDevice.getId()).remove(prevParent.getId());
329
330     List<String> modules =
331     ↳ modulePlacement.getModulesOnPath().get(fogDevice.getId()).get(newParent.getId());
332     modulePlacement.getModulesOnPath().get(fogDevice.getId()).put(newParent.getId(),
333     ↳ modules);
334 }

```

6.2 Dynamic Mobility Controller Code

```

1  @Override
2  protected Map<String, Integer> determineModulesNewDevice(
3      ModulePlacement modulePlacement,
4      List<String> modulesToMigrate,
5      List<Integer> newPath,
6      Map<String, Integer> prevModuleDeviceIds) {
7      if (modulePlacement instanceof ModulePlacementDynamicMobileEdgewards) {
8          return ((ModulePlacementDynamicMobileEdgewards) modulePlacement)
9              .placeMigratingModulesInPath(newPath, modulesToMigrate);
10     } else {
11         return super.determineModulesNewDevice(
12             modulePlacement, modulesToMigrate, newPath, prevModuleDeviceIds);
13     }
14 }
15
16 @Override
17 protected void handleMigrateModulesOnPath(
18     String applicationName, List<Integer> prevPath, List<Integer> newPath) {
19     ModulePlacementDynamicMobileEdgewards modulePlacement =

```

```

20         (ModulePlacementDynamicMobileEdgewards)
21         getAppModulePlacementPolicy().get(applicationName);
22
23     List<String> modulesToMigrateDown = determineIfMigrateModulesDown(prevPath);
24     if (!modulesToMigrateDown.isEmpty()) {
25         int pathId = modulePlacement.getPathId(prevPath);
26         migrateModulesDown(modulesToMigrateDown, pathId, applicationName);
27     }
28
29     List<String> modulesToMigrateUp = determineIfMigrateModulesUp(newPath);
30     if (!modulesToMigrateUp.isEmpty()) {
31         migrateModulesUp(modulesToMigrateUp, newPath, applicationName);
32     }
33 }
34
35 @Override
36 protected void updateNewParent(
37     Map<String, Integer> migratingModules,
38     ModulePlacementMobileEdgewards modulePlacement,
39     int numMigrating,
40     List<Integer> newParentPath,
41     boolean isModuleMigration) {
42     super.updateNewParent(
43         migratingModules, modulePlacement, numMigrating, newParentPath,
44         ↪ isModuleMigration);
45
46     ModulePlacementDynamicMobileEdgewards modulePlacementDynamic =
47         (ModulePlacementDynamicMobileEdgewards) modulePlacement;
48     int pathId = modulePlacementDynamic.getPathId(newParentPath);
49     updateNewPathOnMigration(migratingModules, modulePlacementDynamic, pathId);
50 }
51
52 private void updateNewPathOnMigration(
53     Map<String, Integer> migratingModules,
54     ModulePlacementDynamicMobileEdgewards modulePlacement,
55     int pathId) {
56     for (String moduleName : migratingModules.keySet()) {
57         if ("mService3".equals(moduleName)) continue; // pin mService3 to cloud
58         Integer deviceId = migratingModules.get(moduleName);
59         if (deviceId == null) continue;
60         modulePlacement.getPathToModuleDeviceMap().get(pathId).put(moduleName,
61             ↪ deviceId);
62     }
63 }
64
65 @Override
66 protected void updateOldParent(
67     Map<String, Integer> migratingModules,
68     ModulePlacementMobileEdgewards modulePlacement,
69     int numMigrating,
70     List<Integer> oldParentPath,

```

```

69         boolean isModuleMigration) {
70     super.updateOldParent(
71         migratingModules, modulePlacement, numMigrating, oldParentPath,
72         ↪ isModuleMigration);
73
74     ModulePlacementDynamicMobileEdgewards modulePlacementDynamic =
75         (ModulePlacementDynamicMobileEdgewards) modulePlacement;
76     int pathId = modulePlacementDynamic.getPathId(oldParentPath);
77     if (!isModuleMigration) {
78         updateOldPathOnMobileMigration(migratingModules, modulePlacementDynamic,
79         ↪ pathId);
80     }
81     System.out.println(
82         "Module Placement after migration "
83         + modulePlacementDynamic.getPathToModuleDeviceMap());
84 }
85
86 private void updateOldPathOnMobileMigration(
87     Map<String, Integer> migratingModules,
88     ModulePlacementDynamicMobileEdgewards modulePlacement,
89     int pathId) {
90     FogDevice fogDevice = getFogDeviceById(pathId);
91
92     if (fogDevice.getChildrenIds().isEmpty()) {
93         List<String> moduleList =
94             new
95             ↪ ArrayList<>(modulePlacement.getPathToModuleDeviceMap().get(pathId).keySet());
96         for (String moduleName : moduleList) {
97
98             ↪ modulePlacement.getPathToModuleDeviceMap().get(pathId).remove(moduleName);
99         }
100     }
101 }
102
103 protected List<String> determineIfMigrateModulesDown(List<Integer> path) {
104     ModulePlacementDynamicMobileEdgewards modulePlacement =
105         (ModulePlacementDynamicMobileEdgewards) getModulePlacementPolicy();
106     int pathId = modulePlacement.getPathId(path);
107     int numUsers = getFogDeviceById(pathId).getChildrenIds().size();
108
109     List<String> modules =
110         modulePlacement.getPathToModuleDeviceMap().get(pathId).keySet().stream()
111             .filter(m -> !"mService3".equals(m))
112             .sorted()
113             .toList();
114
115     List<String> modulesToMigrateDown = new ArrayList<>();
116     double totalLoad = 0;
117
118     for (String module : modules) {
119         if (modulePlacement.getPathToModuleDeviceMap().get(pathId) == null

```

```

116         ||
        ↳ modulePlacement.getPathToModuleDeviceMap().get(pathId).get(module)
        ↳ == null) {
117     continue;
118 }
119
120     int currDevice =
121         modulePlacement
122             .getPathToModuleDeviceMap()
123             .get(pathId)
124             .getOrDefault(module, -1);
125     if (currDevice == -1) continue;
126
127     int childId = getChildFromPath(currDevice, path);
128     if (childId == -1) continue;
129
130     totalLoad += modulePlacement.getModuleLoad().get(module) * numUsers;
131
132     FogDevice childDevice = getFogDeviceById(childId);
133     double deviceCpuLoad = modulePlacement.getCurrentCpuLoad().get(childId);
134
135     if (deviceCpuLoad + totalLoad
136         <= childDevice.getHost().getTotalMips() * (0.7 + (threshold *
137             ↳ 0.50))) {
138         modulesToMigrateDown.add(module);
139     }
140
141     System.out.println("Modules we are migrating down " + modulesToMigrateDown);
142     return modulesToMigrateDown;
143 }
144
145 protected void migrateModulesDown(List<String> modules, int pathId, String
146     ↳ applicationName) {
147     Application application = getApplications().get(applicationName);
148     Map<String, Integer> moduleNewDeviceIds = new HashMap<>();
149     Map<String, Integer> moduleOldDeviceIds = new HashMap<>();
150     ModulePlacementDynamicMobileEdgewards modulePlacement =
151         (ModulePlacementDynamicMobileEdgewards) getModulePlacementPolicy();
152     int numInstance = getFogDeviceById(pathId).getChildrenIds().size();
153
154     for (String module : modules) {
155         int prevDevice =
156             ↳ modulePlacement.getPathToModuleDeviceMap().get(pathId).get(module);
157         int newDevice = getChildFromPath(prevDevice,
158             ↳ modulePlacement.paths.get(pathId));
159
160         moduleOldDeviceIds.put(module, prevDevice);
161         moduleNewDeviceIds.put(module, newDevice);
162     }

```

```

161 List<Integer> path = modulePlacement.paths.get(pathId);
162 creatModulesOnDevices(modules, moduleNewDeviceIds, application, -1, false);
163 updateNewParent(moduleNewDeviceIds, modulePlacement, numInstance, path, true);
164
165 List<String> modulesToRemove =
166     determineModulesToRemoveOldParent(
167         moduleOldDeviceIds, modulePlacement, numInstance, path);
168 updateOldParent(moduleOldDeviceIds, modulePlacement, numInstance, path, true);
169 removeModuleFromDevices(
170     modulesToRemove, moduleOldDeviceIds, modulePlacement, application, -1,
171     ↪ false);
172
173 // Mark that a migration was performed during this mobility cycle
174 migrationTriggered = true;
175 }
176
177 protected List<String> determineIfMigrateModulesUp(List<Integer> path) {
178     ModulePlacementDynamicMobileEdgewards modulePlacement =
179         (ModulePlacementDynamicMobileEdgewards) getModulePlacementPolicy();
180
181     List<String> modulesToMigrateUp = new ArrayList<>();
182     for (int i = path.size() - 3; i < path.size() - 1; i++) {
183         int deviceId = path.get(i);
184         FogDevice fogDevice = getFogDeviceById(deviceId);
185
186         double currentCpuLoad = modulePlacement.getCurrentCpuLoad().get(deviceId);
187
188         if (currentCpuLoad
189             >= fogDevice.getHost().getTotalMips() * (0.7 + (threshold * 0.50)))
190             ↪ {
191             List<String> modules = modulePlacement.currentModuleMap.get(deviceId);
192             if (modules == null || modules.isEmpty()) continue;
193
194             List<String> filtered = new ArrayList<>();
195             for (String m : modules) {
196                 if (!"mService3".equals(m)) filtered.add(m);
197             }
198             if (filtered.isEmpty()) continue;
199
200             Collections.sort(filtered);
201             modulesToMigrateUp.add(filtered.getLast());
202         }
203     }
204
205     System.out.println("Module we are migrating up " + modulesToMigrateUp);
206     return modulesToMigrateUp;
207 }
208
209 protected void migrateModulesUp(
210     List<String> modules, List<Integer> path, String applicationName) {
211     Application application = getApplications().get(applicationName);

```

```

210     ModulePlacementDynamicMobileEdgewards modulePlacement =
211         (ModulePlacementDynamicMobileEdgewards) getModulePlacementPolicy();
212
213     int pathId = modulePlacement.getPathId(path);
214     Map<String, Integer> moduleNewDeviceIds = new HashMap<>();
215     Map<String, Integer> moduleOldDeviceIds = new HashMap<>();
216
217     if (modules.size() > 1)
218         System.out.println("If modules to migrate up greater than 2 we are cooked");
219
220     ArrayList<String> modulesToCreate = new ArrayList<>();
221     ArrayList<String> modulesToRemove = new ArrayList<>();
222
223     for (String module : modules) {
224         int prevDevice =
225             ↪ modulePlacement.getPathToModuleDeviceMap().get(pathId).get(module);
226         int newDevice = getFogDeviceById(prevDevice).getParentId();
227
228         moduleNewDeviceIds.put(module, newDevice);
229         moduleOldDeviceIds.put(module, prevDevice);
230
231         Map<String, Integer> tempModuleOldDeviceIds = new HashMap<>();
232         Map<String, Integer> tempModuleNewDeviceIds = new HashMap<>();
233
234         tempModuleOldDeviceIds.put(module, prevDevice);
235         tempModuleNewDeviceIds.put(module, newDevice);
236
237         int numInstance =
238             ↪ modulePlacement.getCurrentModuleInstanceNum().get(prevDevice).get(module);
239
240         List<String> moduleToCreate =
241             determineModulesToCreateOnNewDevice(
242                 tempModuleNewDeviceIds, modulePlacement, numInstance, path);
243         modulesToCreate.addAll(moduleToCreate);
244         createModulesOnDevices(moduleToCreate, moduleNewDeviceIds, application, -1,
245             ↪ false);
246         updateNewParent(tempModuleNewDeviceIds, modulePlacement, numInstance, path,
247             ↪ true);
248
249         List<String> moduleToRemove =
250             determineModulesToRemoveOldParent(
251                 tempModuleOldDeviceIds, modulePlacement, numInstance, path);
252         modulesToRemove.addAll(moduleToRemove);
253         removeModuleFromDevices(
254             moduleToRemove, moduleOldDeviceIds, modulePlacement, application,
255             ↪ -1, false);
256         updateOldParent(tempModuleOldDeviceIds, modulePlacement, numInstance, path,
257             ↪ true);
258     }

```

```

255     System.out.println("We are migrating modules " + modules + " up");
256     System.out.println("We are removing modules " + modulesToRemove);
257
258     Map<String, Integer> filteredModuleOldDeviceIds =
259         createFilteredModuleMap(modulesToRemove, moduleOldDeviceIds);
260     Map<String, Integer> filteredModuleNewDeviceIds =
261         createFilteredModuleMap(modulesToCreate, moduleNewDeviceIds);
262
263     updateAllPathDevices(
264         modulePlacement, pathId, filteredModuleOldDeviceIds,
265         ↪ filteredModuleNewDeviceIds);
266
267     // Mark that a migration was performed during this mobility cycle
268     migrationTriggered = true;
269 }
270
271 private void updateAllPathDevices(
272     ModulePlacementDynamicMobileEdgewards modulePlacement,
273     int pathId,
274     Map<String, Integer> moduleOldDeviceIds,
275     Map<String, Integer> moduleNewDeviceIds) {
276     System.out.println(moduleOldDeviceIds);
277
278     List<Integer> gatewayIds =
279         findAllGateways(
280             ↪ moduleOldDeviceIds.keySet().iterator().next(),
281             new ArrayList<>());
282
283     for (String moduleName : moduleOldDeviceIds.keySet()) {
284         for (int gatewayId : gatewayIds) {
285             if
286                 ↪ (modulePlacement.getPathToModuleDeviceMap().get(gatewayId).containsKey(moduleName))
287                 ↪ {
288                 modulePlacement
289                     .getPathToModuleDeviceMap()
290                     .get(gatewayId)
291                     .put(moduleName, moduleNewDeviceIds.get(moduleName));
292             }
293         }
294     }
295 }
296
297 private List<Integer> findAllGateways(int deviceId, List<Integer> gatewayDeviceIds)
298     ↪ {
299     FogDevice fogDevice = getFogDeviceById(deviceId);
300     if (fogDevice.getLevel() == 2) {
301         gatewayDeviceIds.add(fogDevice.getId());
302     } else {
303         for (int childId : fogDevice.getChildrenIds()) {
304             findAllGateways(childId, gatewayDeviceIds);
305         }
306     }
307 }

```

```

301     }
302 }
303
304     return gatewayDeviceIds;
305 }
306
307 private void updateModulePathDeviceId(
308     ModulePlacementDynamicMobileEdgewards modulePlacement,
309     int pathId,
310     Map<String, Integer> moduleOldDeviceIds,
311     Map<String, Integer> moduleNewDeviceIds) {
312     for (String module : moduleOldDeviceIds.keySet()) {
313         modulePlacement.getPathToModuleDeviceMap().get(pathId).remove(module);
314     }
315
316     for (String module : moduleNewDeviceIds.keySet()) {
317         modulePlacement
318             .getPathToModuleDeviceMap()
319             .get(pathId)
320             .put(module, moduleNewDeviceIds.get(module));
321     }
322 }
323
324 private Map<String, Integer> createFilteredModuleMap(
325     List<String> modules, Map<String, Integer> moduleDeviceIds) {
326     Map<String, Integer> filteredMap = new HashMap<>();
327     for (String module : modules) {
328         if (moduleDeviceIds.containsKey(module)) {
329             filteredMap.put(module, moduleDeviceIds.get(module));
330         }
331     }
332     return filteredMap;
333 }
334
335 private int getChildFromPath(int deviceId, List<Integer> path) {
336     for (int i = path.size() - 1; i > 0; i--) {
337         if (path.get(i) == deviceId) {
338             return path.get(i - 1);
339         }
340     }
341     return -1;
342 }

```

6.3 Path-Aware Module Placement

It is worth noting that throughout the `ModulePlacementDynamicMobileEdgewards` class changes were made to accomodate the new logic. The below code resrepresents the primary method which

implements the path-aware logic.

```
1  @Override
2  protected int isPlacedUpstream(String operatorName, List<Integer> path) {
3
4      int pathId = getPathId(path);
5      for (int deviceId : path) {
6          if (currentModuleMap.containsKey(deviceId)
7              && currentModuleMap.get(deviceId).contains(operatorName)) {
8              for (List<String> list : moduleMapping.getModuleMapping().values()) {
9                  for (String module : list) {
10                     if (module.equals(operatorName)) {
11                         return deviceId;
12                     }
13                 }
14             }
15
16         } else if (pathToModuleDeviceMap.containsKey(pathId)
17             && pathToModuleDeviceMap.get(pathId).containsKey(operatorName)
18             && pathToModuleDeviceMap.get(pathId).get(operatorName) != null) {
19
20             return pathToModuleDeviceMap.get(pathId).get(operatorName);
21         }
22     }
23     return -1;
24 }
```

6.4 Resource-Aware Gateway Selection

```
1  @Override
2  public int determineParent(int resourceId, double time) {
3      if (time <= References.INIT_TIME) {
4          return super.determineParent(resourceId, time);
5      }
6      numDetermineParent++;
7
8      String dataId = getDataIdByInstanceId(resourceId);
9      int resourceLevel = getDataObject().resourceAndUserToLevel.get(dataId);
10     if (resourceLevel != getDataObject().levelID.get("User")) {
11         return super.determineParent(resourceId, time);
12     }
13
14     Location userLoc = getUserLocationInfo(dataId, time);
15
16     String proxyResourceId = getNearestProxyResource(userLoc);
17     Location proxyLocation = dataObject.resourceLocationData.get(proxyResourceId);
18     int proxyBlock = proxyLocation.block;
```

```

19
20     FogDevice mobile = getFogDeviceByID(resourceId);
21     FogDevice prevParent = getFogDeviceByID(mobile.getParentId());
22
23     Map<String, List<AppModule>> appModulesToMigrate =
24         getApplicationModuleListToMigrate(mobile, prevParent);
25     List<AppModule> flat = new ArrayList<>();
26     for (List<AppModule> modules : appModulesToMigrate.values()) {
27         flat.addAll(modules);
28     }
29
30     double moduleLoad = getTotalModulesLoad(flat);
31
32     List<String> gatewayDataIds = getBlockWiseGate(proxyBlock);
33     if (gatewayDataIds == null || gatewayDataIds.isEmpty()) {
34         return mobile.getParentId();
35     }
36
37     final double IDLE_EPS = 1e-9; // treat <= this as idle
38
39     int bestActiveFitId = References.NOT_SET;
40     double bestActiveFitScore = Double.POSITIVE_INFINITY;
41
42     int bestAnyFitId = References.NOT_SET;
43     double bestAnyFitScore = Double.POSITIVE_INFINITY;
44
45     int leastBusyAnyId = References.NOT_SET;
46     double leastBusyAnyScore = Double.POSITIVE_INFINITY;
47
48     List<String> gatewaysName =
49         gatewayDataIds.stream()
50             .map(x -> getFogDeviceByID(dataToInstanceId.get(x)).getName())
51             .toList();
52
53     for (String gwDataId : gatewayDataIds) {
54         Integer gwIdObj = dataToInstanceId.get(gwDataId);
55         if (gwIdObj == null) continue;
56         int gwId = gwIdObj;
57
58         FogDevice gw = getFogDeviceByID(gwId);
59         if (gw == null) continue;
60
61         double used = determineGatewayPathLoad(gw);
62         double cap = gw.getHost().getTotalMips();
63         if (cap <= 0) continue;
64
65         double utilNow = used / cap;
66         double utilAfter = (used + moduleLoad) / cap;
67
68         boolean isIdle = used <= IDLE_EPS;
69         boolean fits = utilAfter <= threshold;

```

```

70         ↪ System.out.println(modulePlacementDynamicMobileEdgewards.getCurrentCpuLoad());
71
72     System.out.println(
73         "\nGW=" + gw.getName() + " id=" + gwId
74         + " used=" + used + " cap=" + cap
75         + " utilNow=" + String.format("%.3f", utilNow)
76         + " utilAfter=" + String.format("%.3f", utilAfter)
77         + " idle=" + isIdle
78         + " fits=" + fits);
79
80     // Track least busy overall (for last resort)
81     if (utilNow < leastBusyAnyScore) {
82         leastBusyAnyScore = utilNow;
83         leastBusyAnyId = gwId;
84     }
85
86     if (!fits) continue;
87     // Tier 2: best fit regardless of idle/active
88     if (utilNow < bestAnyFitScore) {
89         bestAnyFitScore = utilNow;
90         bestAnyFitId = gwId;
91     }
92
93     // Tier 1: best fit among ACTIVE gateways only (not idle)
94     if (!isIdle && utilNow < bestActiveFitScore) {
95         bestActiveFitScore = utilNow;
96         bestActiveFitId = gwId;
97     }
98 }
99
100 if (bestActiveFitId != References.NOT_SET) {
101     System.out.println(
102         "\nChosen ACTIVE gateway (fits): "
103         + getFogDeviceByID(bestActiveFitId).getName());
104     return bestActiveFitId;
105 }
106
107 if (bestAnyFitId != References.NOT_SET) {
108     System.out.println(
109         "\nNo active gateway fits. Chosen gateway that fits (may be idle): "
110         + getFogDeviceByID(bestAnyFitId).getName());
111     return bestAnyFitId;
112 }
113
114 if (leastBusyAnyId != References.NOT_SET) {
115     System.out.println(
116         "\nNo gateway fits threshold. Fallback least-busy overall: "
117         + getFogDeviceByID(leastBusyAnyId).getName());
118     System.out.println("data ids considered " + gatewaysName);
119     return leastBusyAnyId;

```

```
120     }  
121  
122     return mobile.getParentId();  
123 }
```

Acknowledgements

I would like to sincerely thank my supervisor, Liam Murphy, for his invaluable guidance and support throughout this project. His experience greatly helped and he consistently went above and beyond to accommodate my sub par time keeping and always responded promptly to any questions I raised.

I would also like to thank my friends and family who have given me the support in times of stress at various stages throughout the project when I found myself in a dead-end or ever-lasting loops of errors when implementing the project. In paticular I would like to thank Penny who gave me prespective remindng me that I was not doing a western blot and therefore never needed to loose the plot.